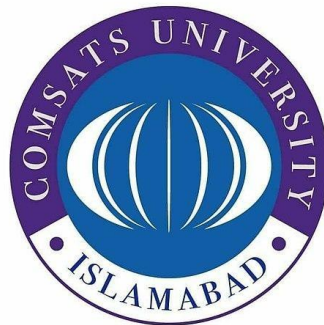


AI-powered Conversational Assistant App

Final Year Project

Session 2022-2026

A project submitted in partial fulfillment of the requirements
for the Degree
of
BS in Computer Science



Department of Computer Science
COMSATS University Islamabad (CUI), Lahore Campus

Project Details

Project ID (for office use)				
Type of project	☒ Traditional ☐ Industrial ☐ Continuing			
Nature of project	☐ Development ☒ Research & Development			
Sustainable Development Goals(SDGs)	☐ Good Health and Well-Being ☒ Quality Education ☐ Industry, Innovation, and Infrastructure ☐ Gender Equality ☐ Decent Work and Economic Growth ☐ Climate Action			
Area of specialization	☒ Artificial Intelligence (AI) ☐ Blockchain ☐ Cybersecurity ☐ Data Science and Analytics ☐ Game Development ☐ Internet of Things (IoT) ☒ Natural Language Processing (NLP) ☒ Mobile App Development ☐ Web Development			
Project Group Members				
Sr.#	Reg. #	Student Name	Email ID	Signature
(i)	FA22-BCS-025	Muhammad Ahmad	gdevaan@gmail.com	
(ii)	FA22-BCS-164	Attique Rehman Alias Adeel	sahitoattique@gmail.com	
Declaration: The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.				

Plagiarism Free Certificate

This is to certify that, I am Muhammad Ahmad S/D/o Umer Farooq, group leader of FYP under registration no CUI/FA22-BCS-025/LHR at the Computer Science Department, COMSATS University Islamabad, Lahore Campus. I declare that my FYP proposal is checked by my supervisor and the similarity index is 3 % that is less than 20%, an acceptable limit by HEC. The report is attached herewith as Appendix A.

Date: 5/6/2026 Name of Group Leader: Muhammad Ahmed Signature: 

Name of Supervisor: Dr. Jawad Shafi Co-Supervisor (if any): _____

Designation: Assistant Professor Designation: _____

Signature:  Signature: _____

HoD: _____

Signature: _____

ABSTRACT

Effective conversation is essential to professional and personal success but still, many of us struggle to hold productive communication. We tend to forget important details, miss-align key concepts, drift away from main agenda and fail to self-reflect during conversation. Existing AI assistants are stateless, treating each conversation independently, they have no recall of previous interactions and no memory. These enterprise assistants are mostly team aligned; they give insights centered around a team rather than individual. To address this, we developed Bubbles, a personal assistant featuring multi-provider inference architecture to help you in your day-to-day interactions. It includes Wingman mode for in-session guidance, Consultant mode that uses hybrid RAG for context-aware Q&A and roleplay system to converse with anyone from past conversations. Furthermore, it comprises Duolingo-inspired gamification to keep users motivated. Evaluations across 30+ test sessions showed that hybrid RAG approach was 95% contextual-aware, wingman advice was under 500ms, and multi-provider fallback architecture ensured zero downtime. With Bubbles isolated interactions are converted into a unified journey of personal growth and assistance.

Table of Contents

Chapter 1. Introduction.....	11
1.1 Overview.....	11
1.2 Problem Statement.....	12
1.3 Proposed Solution.....	13
1.4 Main Objectives.....	13
1.5 Assumptions & Constraints	13
1.6 Project scope.....	14
1.7 Software Development Lifecycle Model	14
Chapter 2. Requirement Analysis.....	15
2.1 Literature Review.....	15
2.1.1 Introduction.....	15
2.1.2 Evolution of Speaker Diarization.....	15
2.1.3 Automatic Speech Recognition (Transcription).....	17
2.1.4 Industry Standards vs. Custom Development	17
2.1.5 Large Language Models in Conversational AI	18
2.1.6 Knowledge Graph based Retrieval Augmentation Generation	18
2.1.7 Real-Time Coaching and Conversational Agents	19
2.1.8 Gamification for Personal Growth.....	19
2.2 Research Gap Identification.....	19
2.2.1 No Real-Time Personal Coaching.....	19
2.2.2 Lack of Context Awareness in Personal Assistants.....	19
2.2.4 Privacy-Focused Personal Tool.....	19
2.2.5 Fragmented Personal Assistant Pipelines.....	19
2.3 Requirements Elicitation.....	20
2.3.1 Functional Requirements	20
2.3.2 Non-functional Requirements.....	20
2.3.3 Requirements Traceability Matrix	20
2.4 Use Case Description.....	21
2.4.1 Use Case 1: Start Live Wingman Session.....	21
2.4.2 Use Case 2: Use Consultant Mode (Context aware Q&A)	22
2.4.3 Use Case 3: View Gamer Center and Performance Dashboard	22
2.4.4 Use Case 4: Roleplay Practice Session.....	22
2.4.5 Use Case 5: Interactive Knowledge Graph Explorer	23
2.4.6 Use Case 6: Session History & Coaching Analysis	23
2.4.7 Use Case 7: Insights Management.....	23
2.4.8 Use Case 8: Voice controls.....	24
2.4.9 Use Case 9: Settings & Personalization.....	24
Chapter 3. System Design/Methodology.....	25
3.1 Activity Diagram	25
3.1.1 Live Wingman Mode	25
3.1.2 Consultant Mode.....	27

3.1.3 Gamification Mode	28
3.1.4 Roleplay Mode.....	29
3.1.5 Interactive knowledge graph.....	29
3.1.6 Session History and Coaching	31
3.1.7 Voice controls.....	31
3.1.8 Insights management.....	32
3.1.9 Settings and Personalization	32
3.2 Software Architecture Diagram	34
3.2.1 Client layer (Flutter and Edge Services)	34
3.2.2 Backend Services and Intelligence Orchestration (FastAPI)	34
3.2.3 Graph-RAG Engine & Data Layer.....	35
3.3 Database Diagram.....	35
3.4 Dataset Details	35
3.4.1 Data Source:.....	35
3.4.2 Data Creation & Processing Pipeline:.....	35
3.4.3 Data Structure & Fields:	36
3.4.4 Dataset Size:.....	36
3.5 Algorithm/Model Selection	37
3.5.1 Automatic Speech Recognition (ASR):	37
3.5.2 Speaker diarization.....	38
3.5.3 Large Language Models (LLM):	38
3.5.4 Semantic Search: Sentence Transformers	38
3.5.5 Contextual Retrieval Algorithm (The "Brain" Logic).....	38
3.5.6 Text to speech.....	39
Chapter 4. Implementation.....	40
4.1 Work Breakdown Structure (WBS)	40
4.2 Team Roles and Responsibilities	41
4.3 Tools and Technologies.....	45
4.3.1 Frontend Development (Mobile Application).....	45
4.3.2 Backend Development (AI Server).....	45
4.3.3 Artificial Intelligence & Machine Learning.....	45
4.3.4 Database & Cloud Infrastructure	46
4.3.5 Development Environment	46
4.4 Implementation Details.....	46
4.4.1 System Architecture Overview	46
Backend "Brain" Implementation (Server-Side).....	48
4.4.3 Real-Time "Wingman" Pipeline Implementation	49
4.5 Screenshots of Prototype.....	51
4.6 Challenges During Implementation	95
4.6.1 Latency Management in Real-Time Advice.....	95
4.6.2 Synchronizing Audio Streams & Speaker Identification	95
4.6.3 Hallucinations in the Consultant Node	95

4.6.4 Managing State in a Stateless Environment.....	95
4.6.5 Mobile Connectivity & Audio Quality.....	96
Chapter 5. Results & Analysis.....	97
5.1 Experimental Setup.....	97
5.1.1 Testing Environment.....	97
5.1.2 Evaluation Metrics Per Subsystem.....	97
5.1.3 Test dataset.....	98
5.2 Results.....	98
5.2.1 Transcription Performance (Groq Whisper).....	98
5.2.2 Speaker Identification (SpeechBrain ECAPA-VoxCeleb).....	98
5.2.3 Wingman Response Latency.....	99
5.2.4 Hybrid RAG Retrieval Performance.....	99
5.2.6 End-to-End Pipeline Summary.....	100
5.3 Comparative Analysis with Existing Work.....	100
5.3.1 Comparative Analysis Table.....	101
5.3.2 Primary function.....	102
5.3.3 Target user.....	102
5.3.4 Real-time coaching.....	102
5.3.5 Identity-aware diarization:.....	102
5.3.6 Cross-session memory.....	102
5.3.7 Retrieval method.....	102
5.3.8 Gamification.....	103
5.3.9 Model architecture.....	103
5.3.10 Privacy model.....	103
5.4 Discussion on Findings.....	103
5.4.1 Multi-Model Architecture: The Right Tool for Each Task.....	103
5.4.2 Hybrid RAG: Why Vector Search Alone Was Insufficient.....	103
5.4.3 Identity-Aware Diarization: From Anonymous Labels to Named Speakers.....	103
5.4.5 Latency Optimization: From 5 Seconds to 500 Milliseconds.....	104
5.4.6 Limitations and Threats to Validity.....	104
Chapter 6. Conclusions.....	105
6.1 Summary.....	105
6.1.1 Project Overview.....	105
6.1.2 Core features.....	105
6.1.3 Technical Architecture.....	105
6.1.4 Results & Evaluation.....	105
6.2 Recommendations for Future Work.....	106
6.2.1 Edge Computing and Offline Execution.....	106
6.2.2 Enhanced Noise Suppression.....	106
6.2.3 Multi-speaker Diarization.....	106
6.2.4 Multilingual Expansion.....	106
6.2.4 Multimodal integration (Video and Facial recognition):.....	106

Chapter 7. References.....107

List of Tables

Table 1: Requirements Traceability Matrix 21

Table 2: Models used..... 39

Table 3: Activity Details 43

Table 4: Evaluation Metrics for Each Subsystem..... 97

Table 5: Groq Whisper Transcription Accuracy 98

Table 6: Voiceprint-Based Speaker Identification at Threshold 0.30 99

Table 7: Wingman End-to-End Latency by Network Condition and Provider..... 99

Table 8: Hybrid RAG Retrieval Study 99

Table 9: Consolidated Results vs Requirements..... 100

Table 10: Comparative Analysis of Bubbles Against Existing Tools 101

List of Figures

Figure 1: Work Breakdown Structure.....	40
Figure 2:Gantt Chart.....	44
Figure 3: Login.....	51
Figure 4: Sign Up	52
Figure 5: Profile Completion.....	53
Figure 6: Onboarding	54
Figure 7: Performa Wizard: Introduction	55
Figure 8: Performa Wizard: Identity.....	56
Figure 9: Performa Wizard: Language and Proficiency	57
Figure 10: Performa Wizard: Goals and Context	58
Figure 11: Performa Wizard: Review	59
Figure 12: Home Screen (Listening State)	60
Figure 13: Home Screen (Server Not Connected).....	61
Figure 14: Home Screen (Server Connected).....	62
Figure 15: Home Screen: Connection Error	63
Figure 16: Entities Sheet	64
Figure 17: Entities Sheet (Empty State)	65
Figure 18: New Session Setup.....	66
Figure 19: New Session: Add Context	67
Figure 20: Live Session: Real-Time Transcription.....	68
Figure 21: Live Session: Mistake Highlight.....	69
Figure 22: Live Session: Controls	70
Figure 23: Session History	71
Figure 24: Session Report	72
Figure 25: Session Report (Analytics).....	73
Figure 26: Consultant Chat.....	74
Figure 27: Roleplay Setup	75
Figure 28: Insights.....	76
Figure 29: Knowledge Graph	77
Figure 30: Knowledge Graph: Node Detail.....	78
Figure 31: Connections.....	79
Figure 32: Game Center	80
Figure 33: Help and Guide	81
Figure 34: Settings: Performa Profile.....	82
Figure 35: Settings: Preferences.....	83
Figure 36: Settings: Preferences Picker.....	84
Figure 37: Settings: Assistant	85

Figure 38: Settings: Assistant Picker	86
Figure 39: Settings: Voice Assistant	87
Figure 40: Settings: Permissions	88
Figure 41: Settings: Data Management	89
Figure 42: Settings: Profile Options	90
Figure 43: Settings: Edit Profile	91
Figure 44: Settings: Help info_1	92
Figure 45: Settings: Help info_2	93
Figure 46: Notifications.....	94

Chapter 1. Introduction

1.1 Overview

Good communication serves as the cornerstone of professional success, academic achievement, and social interaction. The world is becoming more digital and fast-paced, requiring people to be able to articulate concepts clearly, recognize subtleties in tone, and retain important information from spoken conversations. Speaking is still difficult because it happens quickly, and we frequently don't realize our mistakes until the interaction is ended, but writing has greatly improved thanks to sophisticated checking tools. In recent years, artificial intelligence (AI) and natural language processing (NLP) have advanced quite quickly. The way computers comprehend us has evolved because of technologies like Large Language Models (LLM) and Automatic Speech Recognition (ASR). However, most apps nowadays only perform one task at a time, such as transcribing, summarizing, or correcting grammar. They don't collaborate to provide real-time assistance. With the use of cutting-edge deep-learning models and contemporary tools, this project bridges the gap between mobile app development, natural language processing, and human-computer interaction (HCI).

This project is motivated by a typical issue that many professionals and students encounter we often fail to notice minor but crucial elements during talks, and we only become aware of our errors like using the inappropriate tone or stating something incorrectly—only after the interaction is over. In situations like business meetings, interviews, or lectures, missing an important point or sending the wrong message can seriously affect the outcome.

Currently, there are numerous notes taking apps that use AI like Otter.ai, Fathom or Notta. These applications work well for note taking during meetings and extracting key information. But they do not provide an interface where the user can ask questions from the AI and get assistance to make the conversation more beneficial. Not only this but some apps have huge learning curves, missing key information and absurdly large prices. These apps are built with team-first approach hence they do not provide feedback to improve communication skills, and their closed-source nature makes people hesitant to use.

Bubbles tackle all these challenges with very simple implementation. Bubbles is built for 1 user helping with just 1 skill, and that is communication improvement. The app is opensource under Apache-V2 license so community can improve on the application and models used in it. All the models used are public for everyone to change. Core features of the app are Wingman, Consultant, Communication Suggestions, Game Center, Insights (based on entities and knowledge graph) and Pico voice controls. In this way we cut down the costs, reduce learning curve and increase the information extraction to maximum for maximum user assistance in conversation.

The main goal of this project is to build a smart mobile assistant that can capture, analyze, and improve conversations in real time. More specifically, the project aims to:

- Assist 1 user only, with their day-to-day meetings.
- Build a reliable ASR-LLM-KGraph system that uses modern models to answer accurately.
- Keep track of all the user interactions using a knowledge graph.
- Add smart analysis tools that can detect tone, suggest better wording, and explain difficult or technical terms as the conversation happens.
- Use speaker diarization so the system can tell who is speaking, even when there are multiple people in the conversation.
- Create a gamified dashboard with XP, levels, quests and achievements that show progress. Rewards improvement to keep user motivated.

The scope of the project is building a mobile app in Flutter with a Python backend. The app facilitates official, semi-formal, and informal conversations. It is currently limited to English and requires an internet connection and a microphone for the backend to conduct the intensive processing on the cloud.

There are layers in the system. The primary layer manages user interaction; it displays key point summaries and leads to new sessions, either live or consultant. The next layer processes the data by converting speech to text and cleaning the transcript by removing noise and filler words. The main work happens in the AI layer. Here, custom FastAPI layer manages how the data moves between ASR, LLM, database and real-time media components. We use speaker diarization to figure out who is talking and large language models (like Llama, Qwen and Gemini) via Groq and Cerebras to summarize conversations and analyze tone and sentiment. The questions are then sent to hybrid RAG system combining Supabase (PostgreSQL pgvector) for vector *search* and a knowledge graph for structured context retrieval, which are sent to LLM to respond to the user query.

After that, we use a feedback process, in which it identifies grammatical and tone errors, makes recommendations on how to be better and improve further, and stores this data in a user profile for later review. The insights module has features such as the "Replay and Review" option that allows users to review past interactions and see concise, helpful feedback and improve their skills. This project produces a fully working mobile app that gives users a complete set of tools to improve how they communicate and an interface for assistance during meetings. The main results include:

- A highly accurate transcription system that works even when multiple people are talking.
- A summarization feature that pulls out entities and details like names, dates, and action items from long conversations.
- Proper answers to user queries from past conversations.
- Accurate suggestions and answers to questions during live sessions
- An interactive dashboard display trends in speech clarity, vocabulary, and tone.
- Suggestions on how to improve past mistakes.

The goal of this project is to facilitate self-improvement using power of LLMs such as:

- Combining multiple NLP tasks in one place: Instead of offering just summarization or just grammar correction, this system brings together transcription, speaker diarization, sentiment analysis, and helpful feedback in a single workflow.
- Real-time soft skills coaching: The app provides immediate suggestions during conversations, such as better responses or explanations of technical terms.
- Gamified communication improvement: It introduces a new way to track progress using points, levels, and error-frequency statistics, like a habit-building app but focused on communication.

1.2 Problem Statement

Often success depends on our effective communication, but it can be quite difficult to identify and correct errors while speaking. Most AI solutions that exist currently concentrate on transcription or post-meeting notes. And most of the time in conversation that is recorded, it isn't aware of the context or doesn't have enough details to provide good real-time feedback to the user. Current solutions do not provide a system where users can retrieve or chat about people, topics or objects from past conversations. A tailored, privacy-conscious assistant that records conversations and offers real-time guidance on clarity, tone, and contextual search across past entities and conversations is needed. Without the high cost of vendor lock-in or privacy risks, and without burdening users with technical complexity, this method can transform routine interactions into learning opportunities, assisting

users in dynamically improving their communication skills while providing seamless contextual search across past entities and conversations.

1.3 Proposed Solution

So, we propose the development of Bubbles, an AI-powered mobile app that acts as a personal assistant. It uses client-server architecture with a Flutter mobile interface to capture live audio and send it to the FastAPI backend that integrates Large Language Models (LLMs) like Llama (via Groq/Cerebras) and Gemini for natural language processing. Key components include Groq Whisper low-latency and accurate transcription, SpeechBrain ECAPA-TDNN speaker diarization to tell speakers apart, and a hybrid RAG combining knowledge graph (persisted to Supabase) and pgvector semantic search to give context-aware answers from past conversations. To tackle the limitations of simple RAG, we use Knowledge Graph RAG in addition to vectors which have nodes and relations that mimic real-life interactions and can extract minute details at runtime just like a natural human conversation but smarter. By adding a gamified feedback loop with XP, level progression, quests, and achievements, the system provides suggestions on tone and vocabulary like what could have been the better choice, giving users actionable insights to improve their soft skills.

1.4 Main Objectives

This app's primary objective is to develop an AI-powered assistant that provides real-time coaching, context-aware Q&A, roleplay practice, entity-based conversations, and gamified progress tracking. The goals we are attempting to accomplish are:

- To Develop a mobile app using Flutter for live audio capture and interactive frontend.
- To extract key information like tone, suggestions, filler words, and summary of conversation.
- To Create a roleplay mechanism for user to rehearse conversing with past personas.
- Create a gamified dashboard that uses XP, streak, levels, and quests to monitor progress.
- To enable hand-free mode using voice commands.
- To Build a Python backend using FastAPI to manage user data and communication with flutter frontend.
- To manage ASR and LLM via direct SDK integrations (Groq, Cerebras, Google) and enable speaker diarization via Deepgram Nova
- To Create Knowledge Graph RAG implemented via vector search (memory table + pgvector) that uses past conversations for accurate, context-aware responses.
- To Use multiple LLM providers including open-weight models (Llama via Groq/Cerebras) and commercial models (Gemini).
- Assess performance using user engagement, response accuracy and speed, and transcription accuracy.

1.5 Assumptions & Constraints

Assumptions:

- The user has a stable, fast internet connection to connect with the cloud backend.
- The app will be used in moderately quiet environments for accurate ASR performance.
- Conversations will mainly be in English.

Constraints:

- Processing Power: Response speed depends on the providers and internet speed.
- Latency: Network delays can affect real-time feedback.
- Data Privacy: User data must be protected and kept private.

1.6 Project scope

Bubbles' scope includes creating cross-platform flutter application with a Python backend. Real-time speech-to-text transcription, speaker diarization for many speakers, and three modes: Live-wing (support in real time), Consultant (context-aware Q&A using Knowledge Graph RAG), Roleplay mode for practicing conversations by imitating entities (people, organizations, etc.) and Speech-improve (coaching and feedback) are among its key features. User progress will be monitored by a gamification system with XP points, level progression, daily streaks, quests, and achievements tracked server-side. Knowledge Graph visualization and pgvector semantic memory search. Insights dashboard for key events, places, actions, people and organizations. Video analysis and facial recognition are not included in the project, and all significant LLM processing takes place using hosted model endpoints (no offline processing on mobile devices). The application only supports English currently.

1.7 Software Development Lifecycle Model

The project used an iterative and incremental development model, which was ideal for AI and NLP R&D-based projects because it allows for continuous improvements of models and settings based on testing. As technology improves, we can introduce new technologies and algorithms to our projects.

Justification: This approach helped create complex modules, like speaker diarization, text transcription, entity extraction, or RAG system, and be tested separately before being integrated into the main app. Like we switched Neo4j with PostgreSQL relational model and Deepgram with SpeechBrain ECAPA-TDNN for diarization.

Structure: The project went through cycles of planning, requirement analysis, design, implementation, building modules, deployment, testing, validating models, and evaluation. This helped us to catch errors early and incremented features over 300 commits, with each integration tested against accuracy and latency.

Chapter 2. Requirement Analysis

2.1 Literature Review

2.1.1 Introduction

Speaker diarization the concept to find who spoke and when and accurate transcription are foundational to many speech applications that claim to solve these problems. Traditional ASR systems used cascaded acoustic, pronunciation and language models, which are complex and inflexible. Recent industry analyses note that “most ASR systems rely on a combination of legacy systems that are slow, inaccurate, and inflexible,” and advocate fully end-to-end deep learning approaches[12]. At the same time, commercial speech APIs raise concerns of cost, latency and data privacy. This motivates building custom on-premise solutions that leverage the latest research. In particular, by combining modern diarization with open-weight ASR (e.g. Whisper or Conformer-based models), a system can achieve state-of-art accuracy while retaining control over data and allowing domain-specific adaptation.

2.1.2 Evolution of Speaker Diarization

2.1.2.1 Foundational Approaches (Clustering & Embeddings)

Early diarization systems were modular: they detected speech segments (Voice Activity Detection), extracted speaker embeddings for each segment, and then clustered those embeddings. For example, x-vectors[1] – fixed-dimensional embeddings from a speaker-discriminative DNN – became a standard representation. Snyder *et al.* (2018) showed that x-vectors, trained on large datasets, “achieve superior performance” compared to traditional i-vector baselines[2]. In practice, x-vectors from a DNN are extracted frame-wise and pooled per segment, then compared using a backend like PLDA (Probabilistic Linear Discriminant Analysis). Snyder *et al.* (2019) applied this to multi-speaker recordings: they extract x-vectors for short speech segments and compute pairwise PLDA scores, then apply agglomerative hierarchical clustering (AHC) to group segments by speaker[2]. This simple pipeline (x-vectors + PLDA + AHC) achieved state-of-the-art diarization accuracy on benchmarks, significantly reducing error rates in the presence of multiple speakers.

Subsequent work refined the clustering step[3]. Landini *et al.* (2022) proposed **VBx**, which replaces heuristic AHC with a Bayesian Hidden Markov Model over the x-vector sequence[3]. The VBx model clusters speakers in time under a Bayesian framework and has been shown to outperform earlier methods: on standard datasets (CALLHOME, AMI, DIHARD), VBx “achieves superior performance” over previous approaches[3]. In summary, the foundational diarization approach became: VAD → extract DNN x-vectors → model speaker similarity with PLDA → cluster (initially by AHC, later by VBx). These modular systems are robust and reproducible, but they assume one speaker per short segment, so they can fail under heavy overlap.

2.1.2.2 Shift to Deep Learning & End-to-End Models

A key limitation of the modular pipeline is **overlapping speech**. In conversational audio, multiple people often speak simultaneously, but traditional clustering assigns exactly one

speaker per segment. To handle overlap more naturally, researchers developed **end-to-end neural diarization** models that directly output speaker activities over time.

One of the first was EEND (End-to-End Neural Diarization) by Fujita *et al.* (2019). Instead of separate modules, their single neural network takes an acoustic feature sequence and outputs a multi-label sequence indicating which of (a fixed number of) speakers are active at each frame[4]. Crucially, they use a permutation-free training loss that directly optimizes diarization error. A major advantage is that the model can be trained with overlapping speech, and indeed Fujita *et al.* report that their EEND model “explicitly handles overlapping speech” and yields much lower error: on simulated two-speaker mixtures, their EEND achieved a diarization error rate (DER) of 12.28%, compared to 28.77% for a conventional clustering system[4]. This demonstrates that end-to-end models can learn to untangle overlaps when given the right loss function.

Early EEND models assumed a fixed number of speakers. Horiguchi *et al.* (2020) addressed this by introducing **EEND-EDA** (Encoder-Decoder-based Attractors). Here, a self-attentive encoder produces frame-wise embeddings, and an LSTM encoder-decoder generates a set of “attractor” vectors that represent the different speakers[5]. The attractors automatically determine how many speakers to attend to. This makes the model flexible to varying speaker counts. In experiments, Horiguchi *et al.* showed EEND-EDA outperforming plain EEND: for example, in a two-speaker scenario their model reached 2.69% DER on simulated mixtures (vs. 4.56% for vanilla EEND), and in an open-set scenario it achieved 15.29% DER on the CALLHOME dataset versus 19.43% for an x-vector clustering baseline[5]. Thus EEND-EDA approaches the overlap-handling strengths of EEND while adding adaptability in speaker count.

Another hybrid approach is **TS-VAD** (Target-Speaker Voice Activity Detection) by Medennikov *et al.* (2020). TS-VAD combines clustering with neural refinement. First, an initial diarization (e.g. x-vector + AHC) is used to estimate an i-vector for each discovered speaker. Then a neural network takes the original audio features plus all speaker i-vectors as inputs, and outputs frame-wise activity probabilities for each target speaker (i.e. “does speaker 1 speak at time t?” and so on)[6]. This allows the model to revisit the audio and assign overlapping speech properly to multiple speakers. In the challenging CHiME-6 dataset, TS-VAD drastically improved performance: it achieved **over 30% absolute reduction in DER** compared to a strong x-vector baseline[6]. TS-VAD’s strength comes from conditioning on known speakers; it effectively resolves heavy overlaps by checking for the presence of each speaker at every frame.

2.1.2.3 Open-Source Frameworks

The research above has been greatly facilitated by open-source toolkits. Notably, **pyannote.audio** (Bredin *et al.*, 2020) provides a modular, trainable pipeline for diarization[7]. Built on PyTorch, pyannote.audio offers ready-to-use neural components for voice activity detection, speaker embedding, speaker change detection, overlapping speech detection, etc., along with pretrained models. According to the authors, the toolkit “comes with pre-trained models covering a wide range of domains... reaching state-of-the-art performance for most of them”[7]. Researchers can combine and fine-tune these blocks to reproduce classic systems (e.g. x-vector clustering) or build new EEND models. In practice, pyannote.audio has become a popular foundation for diarization research and development, enabling rapid prototyping of custom pipelines.

2.1.3 Automatic Speech Recognition (Transcription)

For transcription, the trend mirrors that in diarization: large neural networks trained at scale can match or beat traditional methods. A breakthrough was **Whisper** (Radford *et al.*, 2023), a weakly-supervised model trained on **680,000 hours** of multilingual audio[8]. Whisper’s massive scale yields remarkable robustness: Radford *et al.* report that their models “generalize well to standard benchmarks” and often rival the best supervised systems without any fine-tuning[8]. By “approaching human accuracy and robustness” on varied speech, Whisper suggests that collecting vast unlabeled data can substitute for carefully curated datasets. In practical terms, Whisper handles diverse accents, noise, and even translates between languages, making it competitive with commercial APIs out-of-the-box.

Alongside data scale, self-supervised learning has advanced ASR. For example, **wav2vec 2.0** (Baevski *et al.*, 2020) learns speech representations by masking segments of raw audio and predicting them via contrastive learning[9]. When fine-tuned on labeled data, wav2vec 2.0 achieves state-of-art results. Strikingly, it excels even with little labeled data: using all 960h of LibriSpeech, it gets ~1.8% WER on clean speech; with only 100h of labels it still outperforms prior methods; and even with just **10 minutes** of labels (plus pretraining on 53,000h unlabeled) it obtains about 4.8% WER on clean speech[9]. This shows that high-quality ASR is possible with limited transcription effort, provided pretraining is leveraged.

Architecturally, modern ASR models use Transformers augmented with convolution. **Conformer** (Gulati *et al.*, 2020) integrates convolutional layers for local feature extraction with Transformer layers for global context[10]. This combination captures both short-term acoustic patterns and long-range dependencies. In experiments on LibriSpeech, Conformer “significantly outperforms” previous models, achieving ~2.1% WER on test-clean without an external language model[10] (and ~3.9% with one). Even a small Conformer variant (10M params) performs competitively[10]. Conformer has become a de facto backbone in many ASR systems (including some commercial products) due to this efficiency and accuracy.

Recent work has also fused ASR and diarization. **WhisperX** (Bain *et al.*, 2023) combines Whisper’s transcription with pyannote’s alignment to produce time-accurate transcripts[11]. In WhisperX, audio is first segmented (via a fast VAD+cut strategy) and fed to Whisper for content. Then forced phoneme alignment (guided by a neural alignment model) refines word-level timestamps. This yields transcripts with precise timing information. WhisperX achieves state-of-the-art results on benchmarks for long-form audio and word segmentation[11], and by batching segments it can run much faster. In effect, WhisperX provides an open-source blueprint: it shows how a robust transcription model and a diarization/aligner toolkit (like pyannote) can be combined for high-performance speech-to-text on long recordings.

2.1.4 Industry Standards vs. Custom Development

Industry leaders have quickly adopted these advances. For example, Deepgram’s engineering blog emphasizes that older hybrid ASR (with separate acoustic/LFMs) is being replaced by fully end-to-end neural networks[12]. Deepgram notes that traditional pipelines make simplifying assumptions that limit performance, and argues that “deep learning is a better approach” for speech recognition[12]. Similarly, AssemblyAI (2022) has trained

Conformer-1 on **650,000 hours** of audio. Their results show this model is *far* more robust than off-the-shelf systems: on a noisy dataset spanning many domains, Conformer-1 makes 43% **fewer** errors on average than other leading models (including OpenAI’s Whisper)[\[13\]](#). This kind of scale (hundreds of thousands of hours) and architectural tuning is often only feasible for commercial teams with abundant resources.

Despite these successes, relying solely on closed APIs has its own drawbacks. Black-box ASR services may perform well generally, but they can struggle in niche conditions. In fact, open models allow domain-specific fine-tuning. For instance, when Whisper is combined with a specialized diarization/alignment pipeline, it can outperform some paid services on certain accents or long audios[\[13\]](#). Additionally, researchers have shown that open ASR models can be customized to local conditions by using data augmentation techniques like noise injection or spectrum masking techniques like SpecAugment [\[14\]](#). In actuality, a custom pipeline can be adjusted based on the speaker population, background noise profile, or target language/dialect, outperforming generic APIs in terms of performance and privacy. Moreover, on-premise solutions eliminate network latency and per-minute costs.

In summary, the literature indicates that modern diarization and ASR can approach or match commercial offerings. By using EEND-EDA and TS-VAD for flexible, overlap-aware diarization, and by employing large open ASR models like Whisper or Conformer, one can build a fully on-premises system with state-of-art accuracy. Frameworks like pyannote.audio and WhisperX[\[15\]](#), provide practical implementations of these ideas. Thus, there is strong evidence that a carefully engineered custom pipeline can achieve “industrygrade” performance while ensuring data sovereignty and tunability.

2.1.5 Large Language Models in Conversational AI

Large Language Models (LLMs) have significantly advanced the field of Conversational AI, enabling more natural and contextually aware interactions[\[16\]](#). Combining the in-context learning abilities of large language models... handles complex dialogues better than intent-based approaches[\[17\]](#). LLMs demonstrate interlocutor awareness by identifying and adapting to dialogue partners[\[18\]](#).

2.1.6 Knowledge Graph based Retrieval Augmentation Generation

Knowledge graphs that represent structural relations between entities have become an increasingly popular research direction [\[19\]](#), (it efficiently) deduce new facts from existing facts[\[19\]](#). It stores human knowledge facts in an intuitive graphical structure. Knowledge graph embedding... represent entities and relations into low-dimensional semantic spaces [\[19\]](#).

LLMs suffer from hallucination... producing confident but factually incorrect content [\[20\]](#). Retrieval-Augmented Generation (RAG) directly addresses this limitation [\[20\]](#). RAG... tightly coupling generation to verifiable, up-to-date evidence [\[20\]](#).

However, traditional RAG systems face limitations because “the complex structure of relationships among different entities... presents challenges for RAG systems”[\[21\]](#). (Knowledge) GraphRAG addresses this by incorporating knowledge graphs, where it “leverages structural information across entities to enable more precise and comprehensive retrieval”[\[21\]](#). This enables systems to utilize relational knowledge, “capturing relational knowledge and facilitating more accurate, context-aware responses”[\[21\]](#). Furthermore, GraphRAG defines a structured workflow, “encompassing Graph-Based Indexing, Graph-

Guided Retrieval, and Graph-Enhanced Generation,” which supports improved reasoning over complex queries[21].

2.1.7 Real-Time Coaching and Conversational Agents

Real-time conversational agents require tightly integrated architectures to enable natural interaction, as modern systems combine components such as “conversational speech recognition... state-action augmented LLM... [and] text-to-speech synthesis” within a unified pipeline[22]. These systems must also operate under strict latency constraints, since delays disrupt conversational flow; achieving human-like dialogue requires responses generated “in close to real time... to comply with human dialogue time constraints”[23]. To address this issue, recent implementations are using sequential, low-latency pipelines. They make use of web-sockets for fast data transfer so user can get feedback in real-time to match natural speed of conversation.

2.1.8 Gamification for Personal Growth

Gamification in non-game apps is crucial to increasing user engagement. An analysis found that organizations with gamified loyalty programs see a 22% increase in customer retention, driving repeat purchases and strengthening brand loyalty[24]. Gamification boosts user engagement by 100%-150% compared to traditional approaches[24].

2.2 Research Gap Identification

Based on the literature review, Bubbles addresses several important gaps:

2.2.1 No Real-Time Personal Coaching

Most existing tools, like Otter or Gong, only analyze a meeting after it has ended. There is no tool in the market that can work as a “Live Wingman,” while giving instant feedback on tone, word choices, or phrasing all while the conversation is still happening without interrupting the flow of the conversation.

2.2.2 Lack of Context Awareness in Personal Assistants

Current assistants such as Siri or Google Assistant or even chat GPT treat each interaction separately and forget previous conversations, as in, it would remember as long as you are in the session. They do not remember professional discussions or build long-term context and do not carry it to the other sessions. Bubbles uses Knowledge Graph RAG alongside pgVectors to track relationships between people and topics across multiple conversations, providing more meaningful, and context-aware information and assistance.

2.2.4 Privacy-Focused Personal Tool

Most enterprise tools are built for companies, which leverage user data to make their products, which compromise individual privacy. There is a need for a lightweight, privacy-conscious app that focuses on personal growth, not organizational monitoring, and all the data control is with the user.

2.2.5 Fragmented Personal Assistant Pipelines

A huge number of tools available today are scattered into individual goals. Each app tries to do one thing only. Wispr Flow only transcribes audio into text, Fathom only summarizes agenda discussed in meetings, Grammarly is text-only and Yoodli lacks context. Bubbles addresses this fragmentation by unifying all these features under one app for a cohesive pipeline.

2.3 Requirements Elicitation

To find out how we can make a great assistant, we used three methods: looking at our own ideas, discussing it with friends as classmates (introspection), studying what's already there in the market, and reviewing documents and publications. We checked top language learning apps like Duolingo and meeting assistants like Otter to figure out the important features and what can be improved upon. In the process the requirements that we gathered are listed below.

2.3.1 Functional Requirements

- FR01 Audio Recording: The app should be able to record audio and transmit it in real-time using the phone's microphone.
- FR02 Real-Time Transcription: The app should be able to turn speech into text in less than a second.
- FR03 Speaker Diarization: The app should be able to tell the difference between the user and other speakers (at least two distinct speakers should be identified).
- FR04 Tone Analysis: The app should check the user's tone and wording to show if it is positive, neutral, or negative.
- FR05 Live Suggestions: In "Wingman" mode, the app should show keyword tips or better word suggestions while talking.
- FR06 Summarization: After a conversation, the app should make a short summary with the agenda, key decisions, and action items.
- FR07 Knowledge Retrieval: In "Consultant" mode, the user should be able to ask about past conversations (for example, "What did I tell Client X last week?").
- FR08 Gamification: The app should give points for completing sessions, maintaining streak and accumulating entities and memories.
- FR09 User Authentication: The app should allow user registration, login and profile management.
- FR10 Entity Chat and Management: The app should allow users to extract any entity from context and chat about the entity and manage relations
- FR11 Coaching Dashboard: The app should show summary, tone, topics, suggestions, and filler words used for each conversation.

2.3.2 Non-functional Requirements

- NFR01 Latency: Suggestions should appear within 2 seconds of speaking.
- NFR02 Accuracy: The speech-to-text system should be correct at least 85% of the time, even with moderate noise.
- NFR03 Scalability: The system should handle many users at the same time without slowing down.
- NFR04 Privacy: User audio data should be safely encrypted both while sending and storing.
- NFR05 Usability: The app should be easy to use. Starting a recording should take no more than 3 taps.
- NFR06 Multi-LLM Fallback: The app should seamlessly switch between Groq, Cerebras, and Gemini LLM providers on failure or rate limits.
- NFR07 Data Persistence: All user data should be persistently stored in Supabase (PostgreSQL + pgvector).

2.3.3 Requirements Traceability Matrix

Req ID	Requirement Description	Module / Component	Priority	Test Case
FR-01	Real-time Audio Recording	Mobile Frontend (Flutter)	High	Audio Capture

Req ID	Requirement Description	Module / Component	Priority	Test Case
FR-02	Real-time Transcription	Backend (Groq/Whisper)	High	Live Text Output
FR-03	Speaker Diarization	AI Module (SpeechBrain ECAPA-TDNN)	Medium	Speaker split
FR-04	Tone/Sentiment Analysis	AI Module (LLM)	Medium	Sentiment detection
FR-05	Wingman suggestions	Frontend (Suggestion widgets), Backend (Wingman service and prompts)	High	Fast and accurate suggestions
FR-06	Session Summary	Backend (Extraction service, Rolling Summarize worker)	Medium	Summary output verification
FR-07	Knowledge Retrieval (RAG)	Database (Supabase/pgvector)	High	RAG query test
FR-08	Gamification (Points/Rewards)	Backend (Gamification Service), Frontend (Game Center)	Medium	XP/Points, Award, Quests
FR-09	User Authentication	Frontend (Auth Screens), Backend (Supabase Auth)	High	Login/Registration Flow
FR-10	Entity Chat and Management	Frontend (Entity Screen) + Backend (Entity/Graph Service)	Medium	Entity Chat & Relation Edit
FR-11	Coaching Dashboard	Frontend (Insights Screen) + Backend (Session Service)	High	Dashboard Data Display
NFR-01	Latency < 2 seconds	System Architecture	High	Latency check
NFR-02	STT Accuracy $\geq 85\%$	Backend (Deepgram)	High	STT accuracy test
NFR-03	Scalability (Multi-user Support)	System Architecture	Medium	Load Test
NFR-04	Privacy (Audio Encryption)	System Architecture	High	Data Encryption Check
NFR-05	Usability (≤ 3 Taps to Record)	Frontend (Flutter)	High	Tap Count Test
NFR-06	Multi-LLM Fallback	Backend (Brain Service)	Medium	Provider Failover Test
NFR-07	Data Persistence (Supabase)	Database (Supabase)	High	Data sync check

Table 1: Requirements Traceability Matrix

2.4 Use Case Description

2.4.1 Use Case 1: Start Live Wingman Session

User Preconditions:

- The user is logged into the application.
- The device has an active internet connection.
- Microphone permissions are granted to the app.

Main Flow:

1. The user selects the "Talk Button" on main screen.
2. The user chooses the conversation style (Formal, Semi-Formal, or Casual).
3. The system begins recording audio via the microphone.
4. The system sends audio chunks to the server for processing.
5. The system displays real-time transcription with speaker diarization.
6. The system shows AI suggestions on the screen, alongside transcription.
7. The user reads the suggestions while talking.
8. The user ends the session when the conversation is over.

Postconditions:

- The audio is saved as local wav file.
- The transcript, speaker diarization, and AI suggestions is saved in database.
- Insights are extracted and saved (highlights, actions, key facts, entities, events)
- Knowledge graphs is updated with newly extracted entities and relations.
- XP is awarded to users for session and quest completion.
- The session appears in history with analytics.

2.4.2 Use Case 2: Use Consultant Mode (Context aware Q&A)

User Preconditions:

- The user has at least one recorded conversation saved in history.
- The knowledge graph is updated with past context.
- Optional: User has entities in the knowledge graph for entity-specific queries.

Main Flow:

1. The user selects "Consultant Mode" from the menu.
2. The user can start new chat or continue old one.
3. The user inputs a text or voice query (e.g., "What was the deadline for the project?").
4. The system uses RAG with vector search and queries knowledge graph to fetch past conversations in memory.
5. The system generates answer using the LLM by passing the context fetched.
6. And then the system displays the answer to the User.

Postconditions:

- After all this, user receives appropriate responses with all basic details.

2.4.3 Use Case 3: View Gamer Center and Performance Dashboard

User Preconditions:

- The user has completed at least one conversation analysis.

Main Flow:

1. When the User navigates to the "Game Center" tab from main dashboard.
2. And then the system retrieves user gamification data: total xp, level and streak
3. The system shows quests based on current level and communication efficiency.

Postconditions:

- The user is aware of their current skill level, frequent mistakes, inappropriate tones, and areas where he can improve improvement. Sa
- They can see their standings among other app users on the leaderboard.
- The user can see personalized improvement suggestions based on performance

2.4.4 Use Case 4: Roleplay Practice Session

User Preconditions:

- The user has at least one person entity in their knowledge graph.

Main Flow:

1. The user navigates to the "Roleplay" mode from main dashboard.
2. The user chooses an entity (person) from knowledge graph
3. The system injects entity context (past conversations, attributes, relations) into LLM prompt.
4. The user converses (using audio or text) with persona, where LLM answers based on context.
5. The user ends roleplay when finished.

Postconditions:

- The roleplay session saved under "roleplay" tag in history.
- The user gains practice of conversing with the entity.
- XP is awarded for completing roleplay session.
- The system may update knowledge graph with new insights from session.

2.4.5 Use Case 5: Interactive Knowledge Graph Explorer

User Preconditions:

- The user has at least one person entity in their knowledge graph.

Main Flow:

1. The user navigates to the "Graph Explorer" mode from main dashboard.
2. The system displays interactive graph with color-based nodes by entity type with edges.
3. The user can tap on any entity to fetch its data and AI summary.
4. The user can use search bar to ask questions about relationships.

Postconditions:

- The User gains visual understanding of their knowledge graph structure.

2.4.6 Use Case 6: Session History & Coaching Analysis

User Preconditions:

- The user has completed at least one conversation.

Main Flow:

1. The user navigates to the "History" tab from main dashboard.
2. The system displays all past sessions (wingman, consultant and roleplay).
3. The user can search for sessions by keywords and tags.
4. The user taps on a session they want to review, and see 3 tabs for that conversation
 - a Analytics: Shows summary, engagement breakdown, sentiment over time and insights.
 - b Coaching: Shows tone, topics, strengths and filler words used.
 - c Playback: Shows full transcription (speakers and ai suggestion) with audio playback.

Postconditions:

- The User can review past conversations and AI suggestions.

2.4.7 Use Case 7: Insights Management

User Preconditions:

- The user has completed at least one conversation.

Main Flow:

1. The user navigates to the "Insights" screen.

2. The system displays all AI extracted insights organized in six categories: All, Events, Key Facts, Actions, Alerts and Updates.
3. The user can search for insights by keyword.
4. The user can edit or delete insights.
5. Quick insights cards is also visible on home screen.

Postconditions:

- The insights help user remember important detail without reading all conversations.
- Edits and deletions are updated in database.
- User can review information extracted from conversations.

2.4.8 Use Case 8: Voice controls

User Preconditions:

- The user has granted microphone permission.
- “Hey bubbles” wake word is enabled in voice assistant settings.

Main Flow:

1. The system listens to wake words in background.
2. The user says "Hey bubbles" to activate voice controls.
3. The user then gives command, examples:
 - a "Hey Bubbles, start a session"
 - b "Hey Bubbles, ask consultant about my meeting"
4. The system parses the voice using ASR for LLM to understand.
5. The LLM then uses tool call to complete the action.

Postconditions:

- The action (session start, consultant, navigation, etc.) is execute successfully.
- The wake word listener goes to idle position, listening for wake word again.
- Voice user is not stored beyond voice command itself for triggering the action only.

2.4.9 Use Case 9: Settings & Personalization

User Preconditions:

- The user is logged in.

Main Flow:

1. The user navigates to “Settings” from the menu.
2. The user can configure multiple things:
 - a General: Theme mode (light/dark/system), accent color, language, quick actions style.
 - b Assistant: Live Wingman tone, Consultant tone, default persona selection.
 - c Voice Assistant: "Hey Bubbles" wake word toggle and voice mode selection.
 - d Data management: Export all your sessions or delete your data from server.
 - e Permissions: View and manage mic, camera, notifications, storage permissions.
3. The user's updated preferences are saved and applied.
4. The user can see information about privacy policy, and terms of service

Postconditions:

- App appearance changes.
- Voice settings updated.
- The user has deleted or exported their personal data.

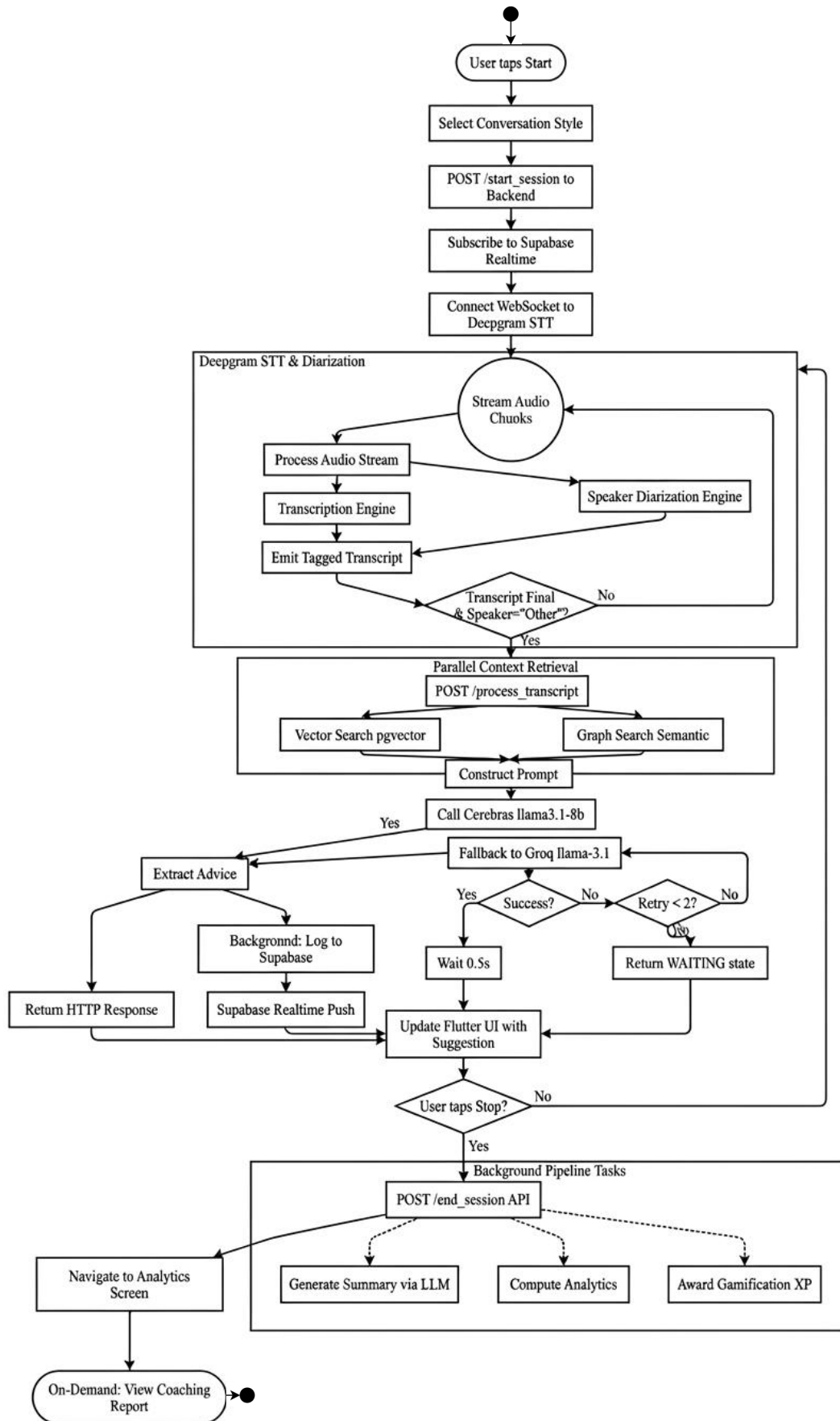
Chapter 3. System Design/Methodology

3.1 Activity Diagram

In the app we have three main Sections:

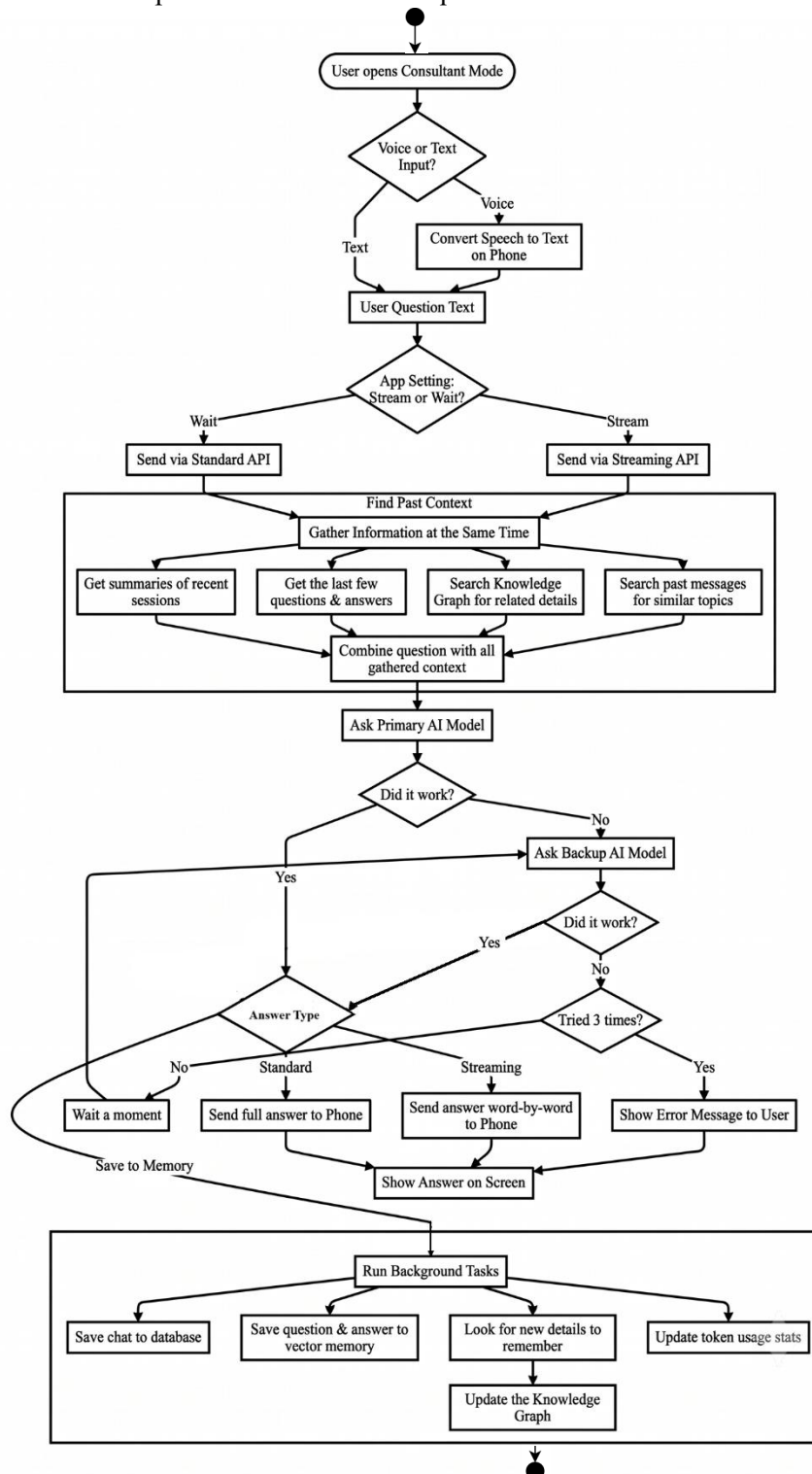
3.1.1 Live Wingman Mode

When the user chooses a conversation style (formal or casual) and begins recording, the process gets started. The system starts a continuous loop in which the audio is streamed via websocket and uses Groq whisper-large-v3-turbo to translate it into text. To create context, it simultaneously retrieves any relevant previous sessions summaries, from the database and associated entities from the Knowledge Graph. Large Language Model (LLM) receives this contextual information and the current discussion in text. Throughout the meeting, LLM continues to show important recommendations and comments on the user's screen. Primary LLM used is Cerebras llama3.1-8b, with Groq llama-3.1-8b-instant as fallback. During and after the live session, user is awarded XP as they achieve their goal. The system saves the audio in local wav format and transcription generated is saved on server. The system then runs in-depth analysis pipeline of the conversation. It includes summary, talk-time metrics, engagement, sentiment and memories and insights overview. It also generates coaching reports that include tone, key topics, suggestions, strengths & filler words.



3.1.2 Consultant Mode

In this mode, the user opens a chat window and starts a new consultant session, then selects text or voice input. The device records the audio and converts it into text using Deepgram, if voice is chosen. Once the text query has been submitted, the system uses a similarity search to find any relevant previous sessions for information. It extracts important people and entities from these sessions. To further add details and make meaningful connections, it then fetches the data from the Knowledge Graph. The user's question is sent to the LLM, along with all the collected information, including nodes, relations, and relevant previous session data. After that, the LLM creates a response, transmits it to the user, and records what happens in the context history. Primary LLM used for consultant mode is gemini-2.5-flash-preview-04-17 with Groq llama-3.3-70b-versatile is fallback.

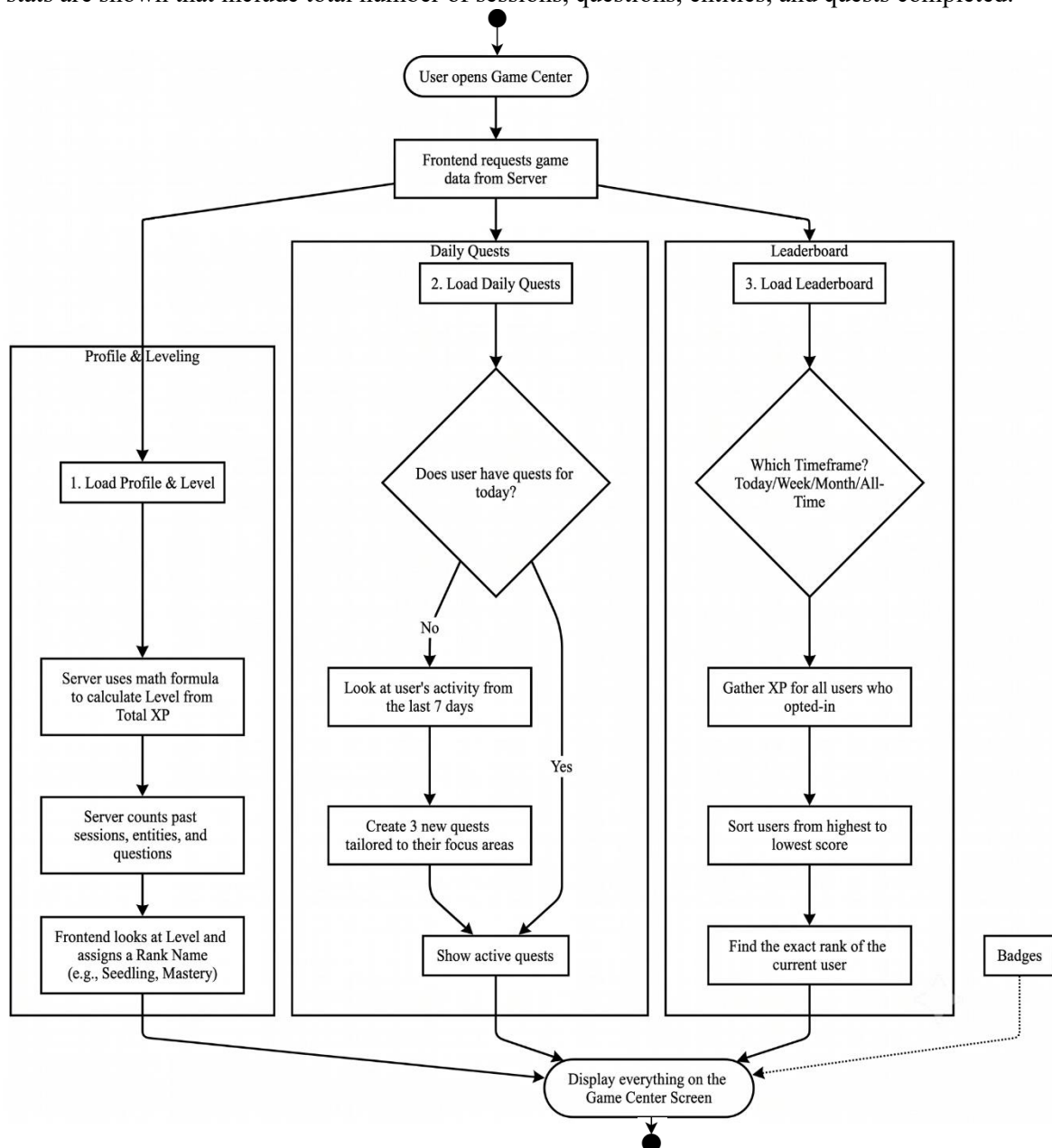


3.1.3 Gamification Mode

This section allows user to see their progress in the gamification module and how they compete with other users on the app on leaderboard. The user can view their XP and level at which they are. At the top user can view their streak. The ranks change based on these level system:

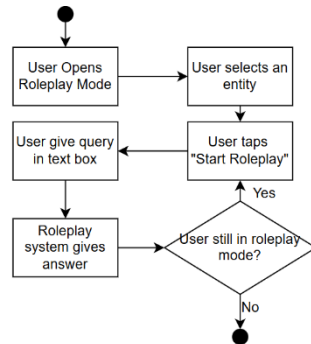
- Level 01 to level 03: Seedling
- Level 04 to level 07: Growth
- Level 08 to level 12: Flourishing
- Level 13 to level 20: Mastery
- Level 21 and above: Legend

Badges are based on achievement user get that include Bronze, Silver, Gold. Furthermore, User is assigned daily quests that they can view on game center to gain maximum XP. At the last, overall stats are shown that include total number of sessions, questions, entities, and quests completed.



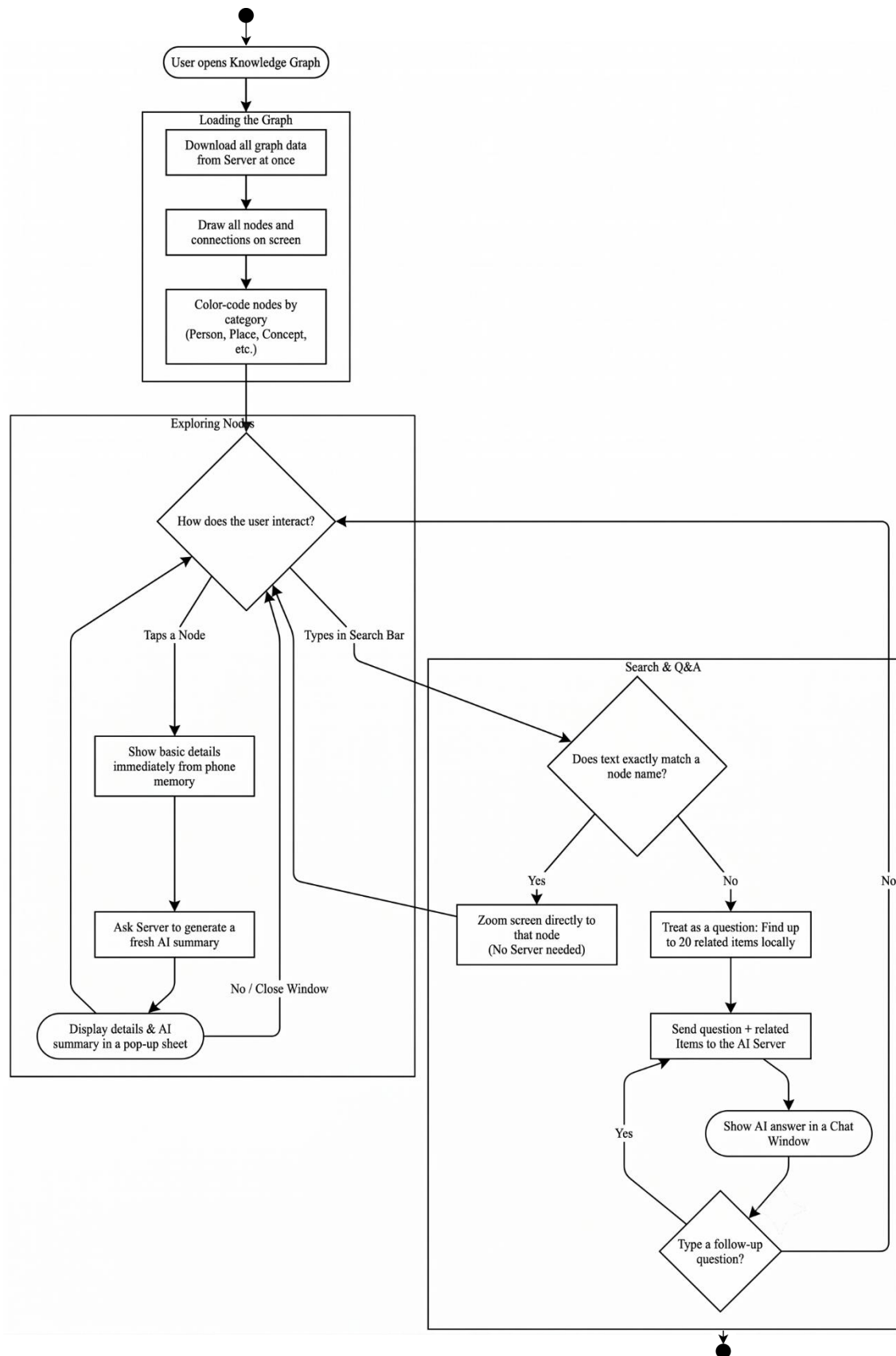
3.1.4 Roleplay Mode

In this section of the app, users can select any entity (person) from their app's memory and converse with them. These entities are created when user is in Live Wingman and Consultant mode.



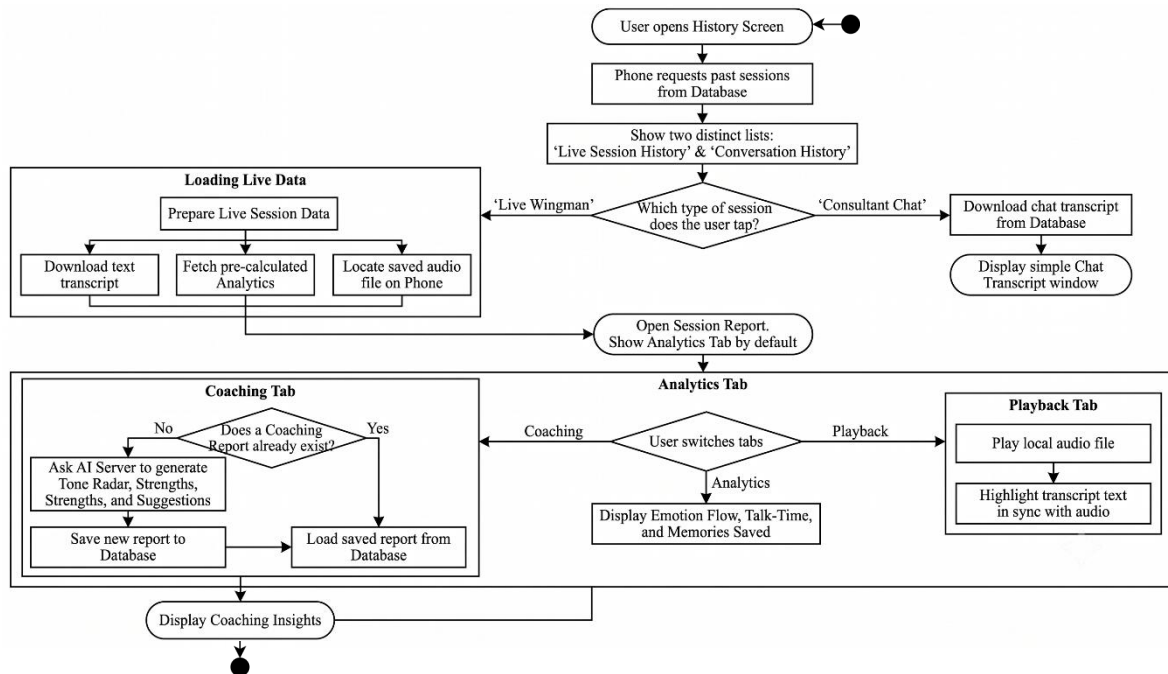
3.1.5 Interactive knowledge graph

In this part of the application, users can view an overall graph of all the entities that exists in their memory. This includes people, places, organizations, events, concepts and objects, each colored differently for better accessibility. User can click and view summary of each entity and relations to other entities in the graph. The user can also use search bar to ask questions regarding entities and their relations to other entities too,



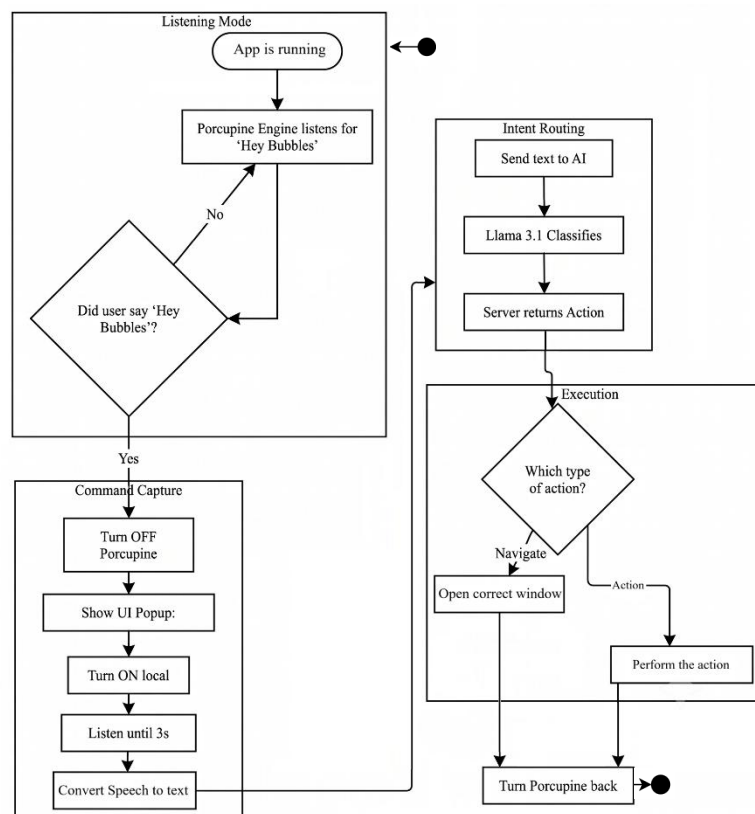
3.1.6 Session History and Coaching

The users can view all their past sessions, that includes all the live Wingman, Consultant, and Roleplay sessions. They can open any session and look into summary and different analytics like sentiment and talk-time ratio. They can also see the Coaching menu for the session to see the mistakes they made, the good things they did and the filler words they used in conversation. They can also play back the exact (local wav) audio and read transcriptions of the sessions too.



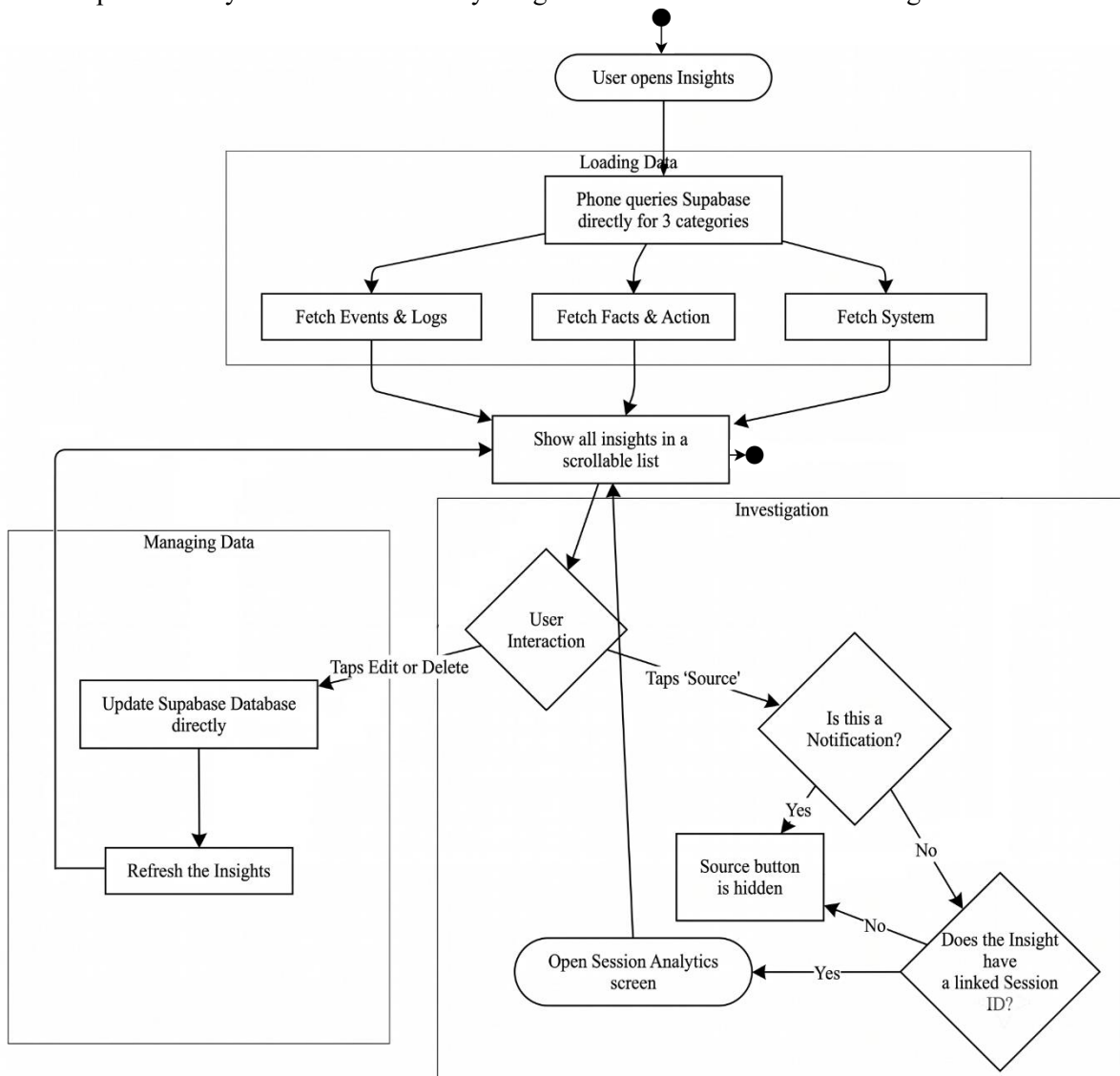
3.1.7 Voice controls

This feature of Bubbles allow user to use the whole app with just voice controls. They users have to just say wake words “Hey bubbles” before any of the task and the system will take care of the rest.



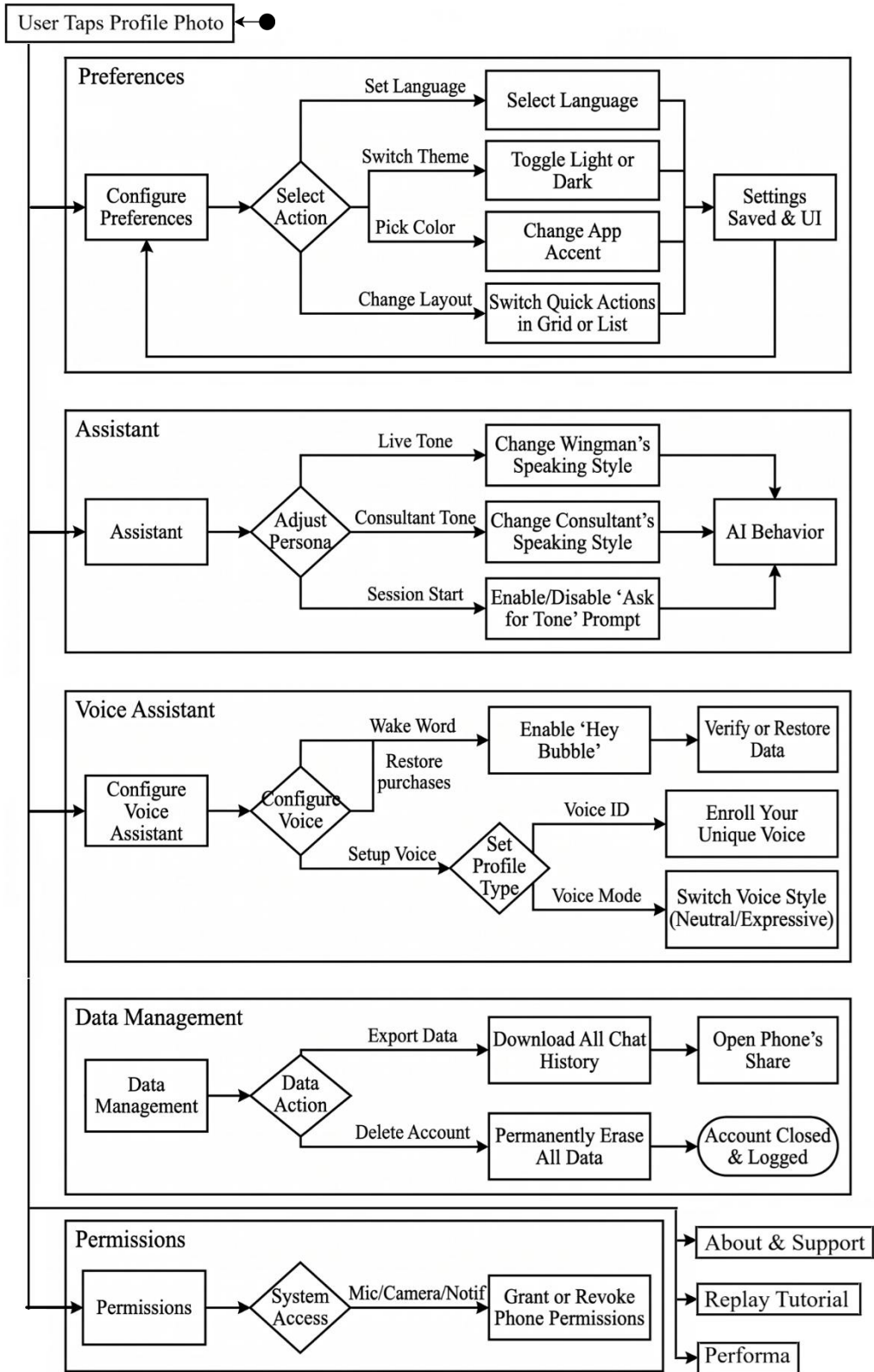
3.1.8 Insights management

The user can use insights management section to quickly look up to meaningful information without going through the conversations. The systems log these types of insights; events, facts, actions, alerts and updates. They can edit or delete any insight and see the source of that insight too.

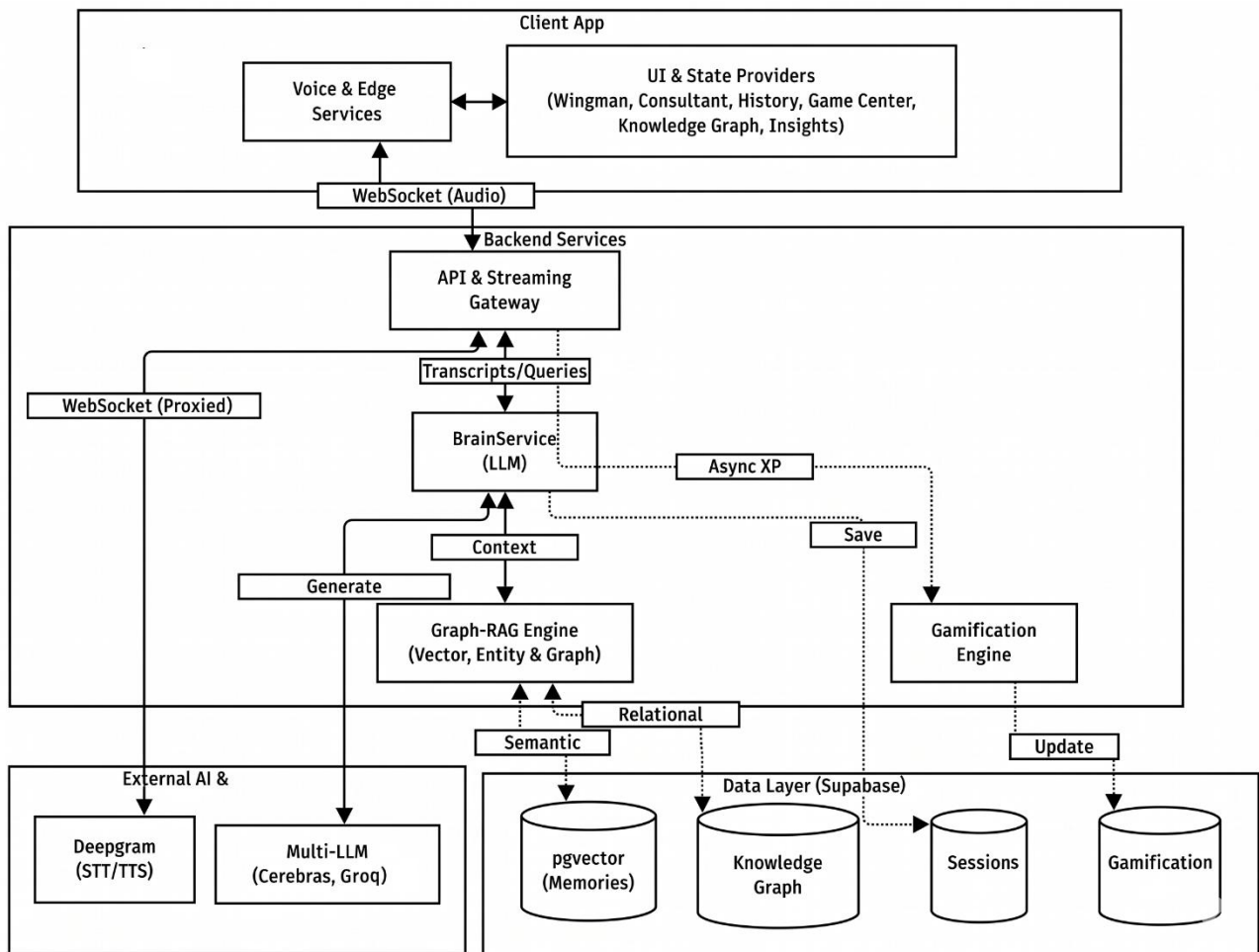


3.1.9 Settings and Personalization

The users can switch to settings menu where they can change different options and preferences for the app. From looks point, they can switch between themes (light and dark), they can change accent colors, language and actions layout. Users can change tone of assistant in both live Wingman and Consultant or choose to ask at start of every session. Furthermore, they can export all their sessions or delete their account too. They can change the permissions allowed to application if user wants to. Lastly, they can investigate project description and contact the developers for feedback.



3.2 Software Architecture Diagram



Bubbles AI make use of multi-modal, real-time architecture to process sessions (Wingman and Consultant) to generate meaningful insights. As show in the diagram above the system is divided into 3 main layers:

3.2.1 Client layer (Flutter and Edge Services)

The flow begins with the mobile application which manages different sections of the app; Wingman, Consultant, Gamification, Insights, Knowledge Graph and History. This layer also handles the edge cases, where it uses PicoVoice for on-device wake-word functionality. For sessions it maintains a direct web-socket connection to backend (custom web socket that uses groq whisper) for real-time transcription and (uses SpeechBrain ECAPA-TDNN) speaker diarization, which ensures lowest latency possible.

3.2.2 Backend Services and Intelligence Orchestration (FastAPI)

Transcripts and user queries reach API and streaming gateway, from where they are forwarded to brain service for further processing. All endpoints are authenticated using Supabase JWT. Brain service, is multi-provider architecture that ensure fallback mechanisms. It uses open-weights models like Cerebras for low-latency live wingman sessions, and Groq for complex reasoning and data extraction. Beyond inference, backend has 14-wroker ARQ (async job queue) for post-processing: entity extraction, session analytics, summaries, etc. These jobs run async on Redis, not blocking API responses.

3.2.3 Graph-RAG Engine & Data Layer

All the data for the app is stored on Supabase using tables and pgvectors. Brain service is ensured to be context-aware with the help of Graph-RAG engine. This hybrid approach make use of cosine similarity and pgvectors to get information from past sessions and while simultaneously fetching entities and their relations to other entities on the knowledge graph. All extracted facts, insights, entities, and session logs are stored in Supabase (PostgreSQL) data layer.

3.3 Database Diagram

3.4 Dataset Details

Unlike traditional machine learning projects that rely on static training datasets, this system utilizes a Dynamic Personal Knowledge Base that is generated in real-time from user interactions and pgvectors embeddings generated from past session transcripts. The system relies on pretrained state-of-the-art models such as Llama 3 (via Groq/Cerebras) for reasoning, Groq Whisper for transcription and constructs a dataset of user context on the fly.

3.4.1 Data Source:

The primary source of data is Real-time User Audio streaming using web socket interface and sent to the backend. This unstructured audio takes two paths:

3.4.1.1 Direct Interaction: Audio input during "Consultant" or "Wingman" sessions, then converted into text to extract entities and relationships.

3.4.1.2 Environmental Context: Active conversations captured to build the user's "memory." To help user remember important details and conversation key points.

3.4.2 Data Creation & Processing Pipeline:

The data undergoes a multi-stage transformation process to become usable for the application; it goes through following processes:

Step 1: Transcription process: Raw audio is processed by Groq Whisper via custom websocket STT. This step includes "Diarization" using SpeechBrain ECAPA-TDNN in which we identify who the main user is and who the others are and filtering out silence or non-speech noise.

Step 2: Input sanitization: Before any processing on the transcription, it is ensured to be stripped of any prompt injection defense. It is normalized into unicode and max character cap of 50k is enforced.

Step 3: Vectorization: The clean text transcripts are passed through a Sentence Transformer model (all-MiniLM-L6-v2) to create 384-dimensional vector embeddings. These embeddings represent the semantic meaning of the conversation.

Step 4: Knowledge Extraction: The text is parallelly processed by multi-provider LLM routers to extract structured knowledge (Entities and Relationships) into a JSON format (e.g., {source: "User", relation: "likes", target: "Coding"}).

Step 5: Sentiment Analysis: Each turn is scored by LLM via a dedicated prompt. The LLM produces a score in between -1 to 1 and a label.

3.4.3 Data Structure & Fields:

The system creates and maintains approximately 40 tables. The tables that directly contribute data structures in the Supabase database:

3.4.4 Dataset Size:

The dataset is dynamic and cumulative. It starts empty as Size = 0; and grows linearly with each user interaction. For testing purposes, the system was validated using 30+ trial sessions, generating approximately 350+ lines of transcript and 45+ extracted graph relationships. And it was able to give exact details, provide help with complete context and provide the relevant information around 80% of the time.

3.4.4.1 Session Logs (*session_logs*): Turn-by-turn record of every conversation:

- id: UUID primary key
- session_id: Reference to the parent session
- turn_index: Integer ordering of the turn within the session
- role: Speaker identity (user, assistant, others, llm, system)
- content: Verbatim transcript text
- speaker_label: Raw diarization label from Deepgram (e.g., "Speaker 1")
- confidence: Transcription confidence score (0.0-1.0)
- sentiment_score: Inline sentiment value (-1.0 to 1.0)
- sentiment_label: Categorized sentiment (positive, negative, neutral)

3.4.4.2 Vector Memory (*memory*): Long-term semantic memory with embeddings for RAG retrieval

- id: UUID primary key
- user_id: Owner of the memory entry
- session_id: Source session where the memory originated
- content: The extracted text fact or observation
- memory_type: Category (general, fact, preference, etc.)
- embedding: 384-dimensional vector (pgvector) for cosine similarity search
- importance: Weight indicating relevance (default 1.0)
- confidence: confidence in memory accuracy (default 1.0)
- source: Origin tag (inferred, explicit, system)

3.4.4.3 Knowledge Graph Nodes (*entities*): People, places, organizations, and concepts extracted:

- id: UUID primary key
- user_id: Owner of the entity
- canonical_name: Normalized, deduplicated name (e.g., "John Smith")
- entity_type: Category (person, place, organization, event, object, concept)
- aliases: Array of alternative names or spellings
- description: Brief summary of what the entity is
- mention_count: Total times the entity has been referenced

3.4.4.4 Knowledge Graph, Edges (*entity_relations*): Relationships between entities:

- id: UUID primary key
- user_id: Owner of the relation

- source_id: UUID of the source entity
- target_id: UUID of the target entity
- relation: Canonical relationship type (e.g., works_at, friend_of, manages)
- strength: Relationship confidence weight (default 1.0)

3.4.4.5 Knowledge Graph, Attributes (entity_attributes): Additional facts about entities:

- id: UUID primary key
- entity_id: Reference to the parent entity
- attribute_key: Fact label (e.g., "age", "department", "location")
- attribute_value: Fact value
- value_type: Data type (string, number, boolean, date)
- confidence: Confidence in the attribute's accuracy

3.4.4.6 Consultant Logs (consultant_logs): Explicit Q&A interactions from the Consultant feature:

- id: UUID primary key
- user_id: Owner of Q&A entry (foreign key to auth.users)
- session_id: Reference to the parent session
- query: Users original question
- question: normalized/cleaned version of query
- responses: AI generated answer (raw)
- answer: Formatted/post-processed version of response

3.4.4.7 Other tables that support core memory and context tables

- User & Identity: profiles, user_settings, user_devices, onboarding_progress. Stores account details, app preferences, registered devices, and onboarding state.
- Session Lifecycle: sessions and session_analytics. Manage session metadata, uploaded audio files, computed per-session statistics, and exported session copies.
- Analytics & Feedback: sentiment_logs, coaching_reports, feedback, app_feedback track per-turn sentiment history, AI-generated coaching summaries, session ratings, and app-level feedback.
- Productivity: highlights, tasks, events. Capture action items, to-do entries, and calendar-style events extracted from conversations.
- Gamification: user_gamification, xp_transactions, quest_definitions, user_quests, achievements, user_achievements. Manage XP, levels, daily quests, and unlockable achievements.

3.5 Algorithm/Model Selection

The main goal for this project is to prioritize latency so that we can get instant responses for real-time feedback and contextual depth for detailed consulting. To achieve this feat, a multi-model architecture is selected, utilizing specific State-of-the-Art models for Audio Diarization, Transcription, and response generation. Note that these modes are placeholders and will be replaced if we find better alternatives.

3.5.1 Automatic Speech Recognition (ASR):

Model Selected: whisper-large-v3-turbo (Groq)

Justification: Whisper was chosen due to its industry-leading speed, which includes low latency in text transcription, timestamp generation, and Voice Activity Detection.

3.5.2 *Speaker diarization*

Model Selected: speechbrain/spkrec-ecapa-voxceleb (ECAPA-TDNN)

Justification: Selected over traditional multi-speaker identification since the app only needs to distinguish between 2 voice prints, user and other speakers. The model is light and loads lazily in worker.

Key Algorithm: During voice enrollment, user records voice sample that is converted to 192-dimensional embedding via SpeechBrain's EncoderClassifier. This voice print is used to detect user when in wingman mode using pgvector's cosine distance. If distance is less than or equal to 0.3 the speaker is identified as user, otherwise labeled as other participants.

3.5.3 *Large Language Models (LLM):*

Model Selected: Llama 3

Justification: Using two different versions of the Llama 3 architecture. Primarily Cerebras is used and with the Groq as fallback. The system performs a two-tiered Inference Strategy such that we have speed when needed and reasoning when necessary.

3.5.3.1 *Real-Time Tier (Wingman Mode & Information Extraction):*

Model: llama-3.1-8b-instant

Justification: Few-Shot Learning algorithm. Because it can produce responses in as little as 50 milliseconds, this 8-billion parameter model is renowned for its speed. It extracts entities (Knowledge Graph Nodes) from transcripts in milliseconds and generates real-time advice.

3.5.3.2 *Deep Reasoning Tier (Consultant Mode):*

Model: llama-3.3-70b-versatile

Justification: The 70-billion parameter model offers superior reasoning capabilities. It is selected for the "Consultant" feature where latency is less critical than the accuracy, depth, and hallucination-resistance of the answer.

3.5.4 *Semantic Search: Sentence Transformers*

Model: all-MiniLM-L6-v2

Algorithm: Cosine Similarity.

Process: This model converts text transcripts into 384-dimensional dense vectors. When a query is made, the system calculates the Cosine Similarity using pgvector HNSW index in Supabase between the query vector and stored memory vectors to retrieve the most semantically relevant past interactions (RAG - Retrieval Augmented Generation).

3.5.5 *Contextual Retrieval Algorithm (The "Brain" Logic)*

A custom Hybrid Retrieval Algorithm was implemented to construct the prompt context:

Vector Search: Retrieves unstructured text logs based graph nodes that are found via embedding similarity.

Graph Traversal: A Neighborhood Search Algorithm is applied to the Knowledge Graph.

Step 1: Key entities are identified in the user's query.

Step 2: The graph traversing algorithm finds uses semantic embedding similarity to find relevant nodes (and immediate neighbors) around these entities to retrieve factual relationships (e.g., User > knows > Asma), covering gaps that vector focus (similarity) might miss.

3.5.6 Text to speech

Model: Edge-TTS.

Justification: Allows for full loop of accessibility for full voice controls. Aids in support for user with visual impairments and hand free use cases.

Component	Selected Model	Purpose
Speech to text	Groq whisper-large-v3-turbo	Voice transcription
Speaker Diarization	SpeechBrain ECAPA-TDNN	Identity of enrolled user via 192-dim embeddings
Fast Inference	Llama 3.1 8B	Wingman mode & entity extraction
Deep Reasoning	Llama 3.3 70B	Complex Consultant Q&A
Embeddings	all-MiniLM-L6-v2	Vectorizing text for long-term memory
Retrieval	Hybrid (pgvector + semantic graph search)	Fetching context from sessions and knowledge graph
Text to speech	Edge-TTS	Accessibility

Table 2: Models used

Chapter 4. Implementation

4.1 Work Breakdown Structure (WBS)

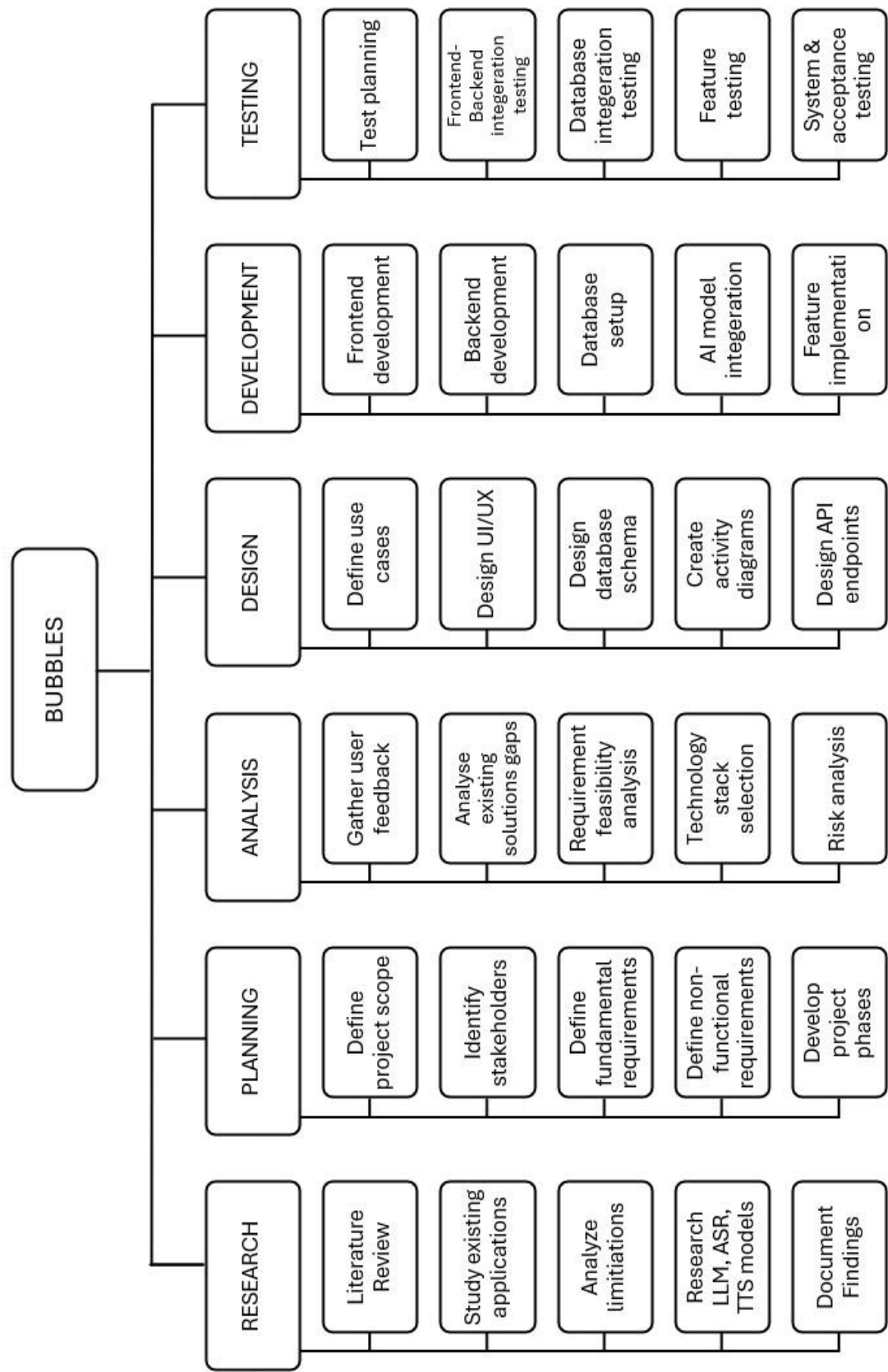


Figure 1: Work Breakdown Structure

4.2 Team Roles and Responsibilities

Team Member	Activity
Phase 1: Architecture & Setup	
M. Ahmad	Designing the high-level System Architecture Diagram (Client-Server-Database).
M. Ahmad	Selecting the Tech Stack (Flutter, Python FastAPI, Supabase) based on requirements.
M. Ahmad	Creating the GitHub Organization and Repository.
M. Ahmad	Setting up .gitignore and Branching rules (Main, Dev, Feature).
M. Ahmad	Configuring the Flutter project structure (Clean Architecture: Providers, Models, Screens).
M. Ahmad	Designing the Relational Database Schema (ERD) for Supabase.
M. Ahmad	Creating SQL tables: users, sessions, messages, memory_logs.
M. Ahmad	Configuring Row Level Security (RLS) policies for data isolation.
M. Ahmad	Setting up Supabase Storage buckets for User Avatars and Audio files.
Phase 2: UI/UX & Design System	
M. Ahmad	Researching modern mobile UI trends (Glassmorphism, Neomorphism).
M. Ahmad	Designing the "Bubbles" Color Palette (Primary, Secondary, Accent colors).
M. Ahmad	Selecting and integrating Google Fonts (Inter, Outfit) for typography.
M. Ahmad	Creating reusable AppButton and AppInput widgets with animations.
M. Ahmad	Designing the custom GlassContainer widget for the frosted glass effect.
M. Ahmad	Implementing Lottie Animations for loading states and empty data states.
M. Ahmad	Designing the application Logo and App Icon variants (Adaptive Icons).
M. Ahmad	Creating the Splash Screen with animated logo reveal.
M. Ahmad	Implementing Dark Mode & Light Mode Theme logic.
Phase 3: Frontend Feature Development	
M. Ahmad	Implementing the AuthProvider for managing User Sessions state.
M. Ahmad	Developing the Login Screen with Form Validation (Email/Password RegEx).
M. Ahmad	building the Signup Screen with secure Password Visibility toggles.
M. Ahmad	Implementing the "Verify Email" OTP screen logic.
M. Ahmad	Creating the App Drawer (Sidebar) with navigation routing.
M. Ahmad	Developing the Main Dashboard Grid layout for features.
M. Ahmad	Building the "Live Wingman" screen with microphone controls.
M. Ahmad	Creating the "Chat Interface" with dynamic bubble message rendering.
M. Ahmad	Implementing Auto-Scroll logic for the chat window.
M. Ahmad	Developing the "History" list view with pagination support.
M. Ahmad	Building the "Settings" screen for User Profile management.
M. Ahmad	implementing "Error Toasts/Snackbars" for network failure feedback.
Phase 4: Integration & Logic (Upcoming/Ongoing)	
M. Ahmad	Integrating supabase_flutter SDK for Auth and Database IO.
M. Ahmad	Writing ApiService class for typed HTTP requests/responses.
M. Ahmad	Integrating LiveKit Client for WebRTC Audio Streaming.
M. Ahmad	Implementing Background Audio permissions for Android/iOS.

Team Member	Activity
M. Ahmad	Handling WebSocket Data Channels for receiving real-time advice.
M. Ahmad	Implementing Local Storage (shared_preferences) for caching settings.
M. Ahmad	Optimizing images using cached_network_image provider.
Phase 5: Release Engineering (Upcoming)	
M. Ahmad	Configuring AndroidManifest.xml permissions (Mic, Internet, Foreground Service).
M. Ahmad	Generating KeyStore for Android App Signing.
M. Ahmad	Optimizing App Bundle size (ProGuard/R8 obfuscation).
M. Ahmad	Creating Screenshots and Graphic Assets for the Play Store listing.
M. Ahmad	Recording and Editing the Final Project Demo Video.
Phase 1: AI Research & Backend Environment	
Attique Rehman	Researching Open Weight LLM models (Llama 3, Mistral, Gemma) for reasoning.
Attique Rehman	Evaluating ASR options (Deepgram, Whisper, Google) for speed vs cost.
Attique Rehman	Installing Python 3.9+ and setting up Virtual Environment (venv).
Attique Rehman	Managing dependencies via requirements.txt,
Attique Rehman	Setting up the FastAPI framework structure (main.py, routers/).
Attique Rehman	Configuring uvicorn server for asynchronous request handling.
Attique Rehman	Implementing CORS Middleware for secure API access.
Attique Rehman	Setting up pydantic-setting BaseSettings for secure Environment Variable management.
Attique Rehman	Creating Pydantic Models for Request/Response Schema validation.
Phase 2: Core Intelligence Pipeline	
Attique Rehman	Integrating Deepgram Python SDK for streaming transcription.
Attique Rehman	Implementing "Speaker Diarization" identifying 'User' vs 'Other speakers'.
Attique Rehman	Integrating Groq and Cerebras Client for ultra-fast Llama 3 inference.
Attique Rehman	Designing the multi-persona system prompts (formal, non-formal, casual, e.t.c).
Attique Rehman	Implementing Context Window management (truncation logic).
Attique Rehman	Storing session with Redis and in-memory fallback support.
Attique Rehman	Handling LLM Rate Limits and Retry logic.
Attique Rehman	Developing the "Wingman" logic for unsolicited advice generation.
Attique Rehman	Implementing rate limiting for API endpoints.
Attique Rehman	Implementing JWT authorization for user and resource ownership.
Attique Rehman	Implementing multi-provider LLM clients (Groq, Cerebras, Gemini) with goal based routing and fallback logic.
Phase 3: Advanced Memory Systems	
Attique Rehman	Integrating sentence-transformers(all-MiniLM-L6-v2) library.
Attique Rehman	Writing logic to chunk text transcripts for embedding.
Attique Rehman	Implementing vector database insertion logic for Supabase via pgvector.
Attique Rehman	Developing Cosine Similarity search algorithms for memory retrieval.
Attique Rehman	Integrating NetworkX library for graph data structures with Supabase persistence.
Attique Rehman	Developing entity extraction pipelines ("Extract Person, Place, Thing").

Team Member	Activity
Attique Rehman	Implementing graph context searching using semantic similarity, keyword fallback, and neighboring nodes traversal.
Attique Rehman	Writing the "Consultant" endpoint combining session history, knowledge graph, vectors, session summaries, and entity context.
Phase 4: Optimization & Refinement	
Attique Rehman	Implementing "Graph-RAG" logic to merge vector/graph contexts.
Attique Rehman	Optimizing Server-side threading for parallel processing.
Attique Rehman	Creating Background Workers for non-blocking database writes.
Attique Rehman	Writing automated scripts to clean up old session logs.
Attique Rehman	Dockerizing the Backend Application for containerized deployment.
Attique Rehman	Developing game center for XP, levels, streak tracking, quests and achievements.
Attique Rehman	Developing Entity Service for entity persistence, fuzzy name matching, and relation alias normalization.
Attique Rehman	Integrating Ngrok for local public URL tunneling during development
Attique Rehman	Hosting the Backend on a cloud instance (e.g., RunPod/AWS).
Phase 5: Validation & Thesis (Upcoming)	
Attique Rehman	Conducting load testing for concurrent users.
Attique Rehman	Measuring "Glass-to-Glass" latency via high-speed logs.
Attique Rehman	Calculating "Hallucination Rate" metrics.
Attique Rehman	Writing the "Methodology" chapter: "Hybrid Memory Architecture".
Attique Rehman	Documenting API endpoints using Swagger UI.

Table 3: Activity Details

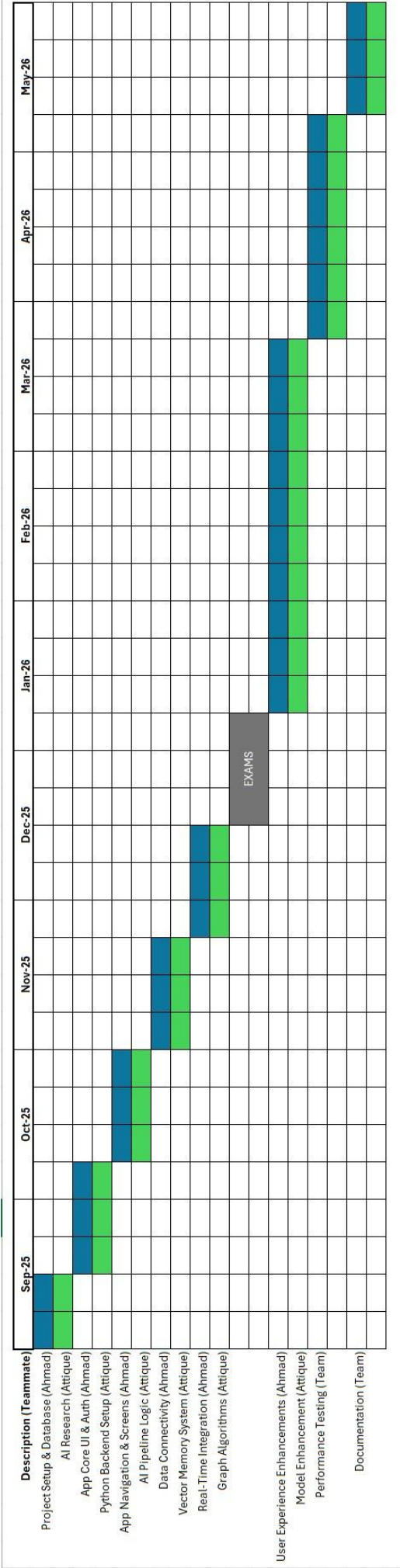


Figure 2:Gantt Chart

4.3 Tools and Technologies

The project was developed using a split-stack architecture, utilizing a cross-platform mobile frontend and a high-performance Python backend for AI processing.

4.3.1 Frontend Development (Mobile Application)

4.3.1.1 *Framework: Flutter (Dart SDK 3.10+)* - Chosen for its single codebase capability for both Android and iOS.

4.3.1.2 *Language: Dart* - The programming language used for the Flutter framework.

4.3.1.3 *Key Libraries:*

- `livekit_client`: For handling real-time WebRTC audio streaming.
- `provider`: For state management across the application.
- `supabase_flutter`: For managing user authentication and database realtime connections.
- `flutter_animate`: For UI animations and transitions.
- `flutter_markdown_plus`: To render rich text responses from the AI.

4.3.2 Backend Development (AI Server)

4.3.2.1 *Language: Python 3.12+* - Selected for its robust support for AI/ML libraries.

4.3.2.2 *Framework: FastAPI* - A modern, high-performance web framework for building APIs with Python 3.9+.

4.3.2.3 *Server Implementation:* `uvicorn` (ASGI server) running asynchronously to handle multiple concurrent audio streams.

4.3.2.4 *Graph Processing:* `Jinja` used to build and traverse the in-memory personal Knowledge Graph.

4.3.2.6 *Background jobs:* For async processing background job queues are created using `arq` >= 0.26 + `redis[hiredis]` >= 5.2.

4.3.2.7 *Tunneling: Ngrok* - Used to expose the local Python backend to the internet for the mobile app to connect.

4.3.3 Artificial Intelligence & Machine Learning

4.3.3.1 *Inference Engine: Cerebras and Groq* - Used as the LPU (Language Processing Unit) to run Llama 3 models with ultra-low latency.

4.3.3.2 *Large Language Model: Llama 3 (via Groq and Cerebras)* - Used for intelligence, reasoning, and text generation. Multi-provider with automatic fallback support.

4.3.3.3 *Speech-to-Text: GroqWhisper model whisperp=large-v3-turbo* - Used for real-time transcription and speaker diarization.

4.3.3.4 *Vector Embeddings: Sentence Transformers (all-MiniLM-L6-v2)* - Python library used to convert text into vector embeddings for semantic search.

4.3.4 Database & Cloud Infrastructure

4.3.4.1 *Primary Database: Supabase (PostgreSQL)* - Used for storing user profiles, structured logs, and session data.

4.3.4.2 *Vector Database: pgvector (on Supabase)* - Used to store high-dimensional vector embeddings for the AI's long-term memory.

4.3.4.3 *Real-time Infrastructure: Custom WS* - A custom web socket infrastructure used to transport low-latency audio between the mobile app and the Python server.

4.3.4.4 *Sessions Store: Redis* - Used for persistent storage for session with in-memory fallback.

4.3.5 Development Environment

4.3.5.1 *IDE*: Visual Studio Code, Antigravity and Claude Code.

4.3.5.2 *API Testing*: Postman (for REST API) and pytest for integration testing.

4.3.5.3 *Version Control*: Git & GitHub.

4.4 Implementation Details

The implementation of the system is divided into three core modules: the Frontend Application (User Interface), the Backend Brain (Intelligence core), and the Real-Time Communication Layer.

4.4.1 System Architecture Overview

With mobile devices with the app serving as input and output terminals and the Python server serving as the central "Brain," for our system the system uses a hub-and-spoke architecture.

4.4.1.1 *Audio Pipeline*: The mobile application records unprocessed audio from mobile's microphone and then transmits it to the server over custom websocket.

Pseudocode:

```
FUNCTION start_audio_streaming():
    INITIALIZE recorder WITH sampling_rate = 16000, encoder = PCM16
    INITIALIZE socket =
CONNECT_WEBSOCKET("wss://brain.bubbles.ai/v1/stt/stream")

    ON recorder RECEIVES audio_bytes:
        IF NOT is_muted:
            socket.SEND_BINARY(audio_bytes)
            audio_buffer.APPEND(audio_bytes)

// [Backend: Audio Stream Handler]
FUNCTION handle_audio_stream(websocket, user_id):
    INITIALIZE buffer AS BYTE_ARRAY
```

```

WHILE TRUE:
    message = websocket.RECEIVE()

    IF message.type IS BINARY:
        buffer.EXTEND(message.bytes)

        IF LENGTH(buffer) >= 192000: // ~12 seconds of audio
            transcript = TRANSCRIBE_AUDIO(buffer)
            websocket.SEND_TEXT({"type": "final", "text": transcript})
            buffer.CLEAR()

```

4.4.1.2 *Cognitive Pipeline*: In order to produce a accurate and useful response, the server processes the audio, retrieves context (Memory + Graph), and requests the LLM (Groq) to generate an appropriate response

Pseudocode:

```

// [Backend: Context Builder]
FUNCTION build_cognitive_context(user_id, transcript, budget_seconds =
0.5):
    TRY:
        // Run all context gathering in parallel within a timeout
        START TIMED_EXECUTION (limit = budget_seconds):

            PARALLEL EXECUTE:
                // 1. Semantic Memory Search
                vector = EMBED_TEXT(transcript)
                memories = SEARCH_SIMILAR_MEMORIES(user_id, vector, limit = 5)

                // 2. Entity & Relation Extraction
                entities = MATCH_GRAPH_NODES(user_id, transcript)
                facts = GET_GRAPH_NEIGHBORHOOD(user_id, entities)

            WAIT FOR PARALLEL TASKS TO COMPLETE
            RETURN NEW WingmanContext(memories, facts, persona)

    CATCH TimeoutError:
        RETURN EmptyContext() // Fallback to prevent latency spikes

```

4.4.1.3 *Feedback Loop*: The app receives the response as either text or virtual speech and then diasplays it for the user to see.

Pseudocode:

```

// [Frontend: Transcript Handler]
FUNCTION on_transcript_received(transcript, speaker):
    IF speaker == "Other":
        // Fire advice request to server
        advice = REQUEST_WINGMAN_ADVICE(transcript, session_id)

    IF advice IS NOT NULL:
        session_logs.APPEND({
            "speaker": "Wingman",
            "text": advice,
            "isSuggestion": TRUE

```

```

    })
    current_suggestion = advice
    UPDATE_USER_INTERFACE() // Triggers UI build

```

Backend "Brain" Implementation (Server-Side)

The backend logic that is currently implemented in server.py is implemented using Python and FastAPI and is structured in four distinct services that run concurrently to provide a seamless experience and are:

4.4.2.1 The Graph Service (Structure Knowledge):

Logic: The NetworkX library is used to implement it, so we can retrieve relations and entities instantly.

Function: It keeps track of each active user's dynamicly in-memory Knowledge Graph. The Llama 3 model is used by the system to extract entities such as "Alice" and "Project X" when a user talks. These entities are included in the graph as nodes. In order to give the AI factual context, the system then searches the graph for direct relationships e.g., Who is Alice? -> Alice is Project Manager. So as a result the response include hard facts and figures.

Pseudocode:

```

CLASS GraphService:
    FUNCTION find_context(user_id, text):
        graph = active_graphs[user_id]

        // Semantic mapping of query text to graph nodes
        query_vector = ENCODE_TEXT(text)

        matched_nodes = EMPTY_LIST
        FOR EACH node IN graph.nodes:
            IF COSINE_SIMILARITY(query_vector, node.vector) > 0.3:
                matched_nodes.APPEND(node)

        facts = EMPTY_LIST
        FOR EACH edge (u, v, data) IN graph.EDGES(matched_nodes):
            relation = data.GET("relation", DEFAULT = "related to")
            facts.APPEND("Fact: " + u + " " + relation + " " + v)

        RETURN FIRST 5 ELEMENTS OF facts

```

4.4.2.2 The Vector Service (Long-Term Memory):

Logic: SentenceTransformer and Supabase pgVectors are used to implement this.

Function: A 384-dimensional vector is "embedded" in each chat log. In order to enable the app and LLM "remember" historical details, the system runs a Cosine Similarity Search against the Supabase database whenever a new query is received.

Pseudocode:

```

CLASS VectorService:
    FUNCTION search_memory(user_id, query):
        // Convert text to high-dimensional vector
        embedding = ENCODE_TEXT(query).TO_LIST()

        // Query Database via similarity search
        results = DATABASE_RPC_CALL("match_memory", {
            "query_embedding": embedding,
            "match_threshold": 0.5,
            "match_count": 3,
            "p_user_id": user_id
        })

        RETURN EXTRACT_CONTENTS(results)

```

4.4.2.3 The Brain Service (Reasoning Core):

Logic: It creates a "System Prompt" for the LLM on the fly. This prompt is not static, before being transmitted to LLM, it is included with Graph Facts and Vector Memories. This guarantees that the model responds using the user's unique information rather than merely generic training data.

Pseudocode:

```

FUNCTION build_reasoning_prompt(user_id, context_data):
    persona = GET_PERSONA(user_id)

    // 1. Base Persona Block
    prompt = RENDER_TEMPLATE("persona_block", persona)

    // 2. Inject Context
    IF context_data.memories IS NOT EMPTY:
        prompt = prompt + "\nPAST CONTEXT:\n" +
JOIN_STRINGS(context_data.memories)

        IF context_data.facts IS NOT EMPTY:
            prompt = prompt + "\nKNOWN FACTS:\n" +
JOIN_STRINGS(context_data.facts)

    // 3. Add Reasoning Instructions
    prompt = prompt + "\nProvide one short, actionable sentence of
advice."

    RETURN prompt

```

4.4.3 Real-Time "Wingman" Pipeline Implementation

The core feature of the project, the "Wingman," requires a specialized non-blocking pipeline to achieve real-time performance and we achieved it by following:

4.4.3.1 *Ingestion*: The server listens to the opens websocket to get audio and then we further process that audio.

Pseudocode:

```

FUNCTION stt_stream_listener(websocket, token):
    user_id = VERIFY_TOKEN(token)
    websocket.ACCEPT_CONNECTION()

```

```

WHILE TRUE:
    message = websocket.RECEIVE_MESSAGE()

    IF message CONTAINS "bytes":
        PROCESS_AUDIO_CHUNK(message["bytes"])
    ELSE IF message CONTAINS "text":
        IF message.text == '{"op": "flush"}':
            TRIGGER_TRANSCRIPTION()

```

4.4.3.2 *Diarization*: Using the Groq Whisper, the incoming audio stream is split by speaker. The system isolates the voice of the "interlocutor" the person talking to the user and the user.

Pseudocode:

```

FUNCTION handle_metadata(json_response):
    words = json_response["channel"]["alternatives"][0]["words"]

    IF words IS NOT EMPTY:
        // Speaker identification from word metadata
        speaker_id = words[0].GET("speaker", DEFAULT = 0)

        // Map ID to persona (0 is usually the primary user)
        IF speaker_id == 0:
            current_speaker = "user"
        ELSE:
            current_speaker = "other"

        IF current_speaker == "other":
            TRIGGER_WINGMAN_LOGIC(current_transcript)

```

4.4.3.3 *Parallel Processing*: The transcript of the interlocutor is processed in parallel:

Process A: Sent to specific vector storage to update context or to get the context that is required to generate LLM response.

Process B: Sent to the 8B Parameter "Fast LLM" to check if advice is needed and if yes then send this back to mobile phone

Pseudocode:

```

FUNCTION process_transcript_wingman(request):
    // FAST PATH: Return advice to user immediately
    advice = GET_ADVICE(request.transcript)

    // SLOW PATH: Offload background tasks
    ENQUEUE_BACKGROUND_JOB("extract_knowledge", request.transcript)
    ENQUEUE_BACKGROUND_JOB("compute_embeddings", request.transcript)
    ENQUEUE_BACKGROUND_JOB("rolling_summarize", request.session_id)

    RETURN WingmanResponse(advice)

```

4.5 Screenshots of Prototype

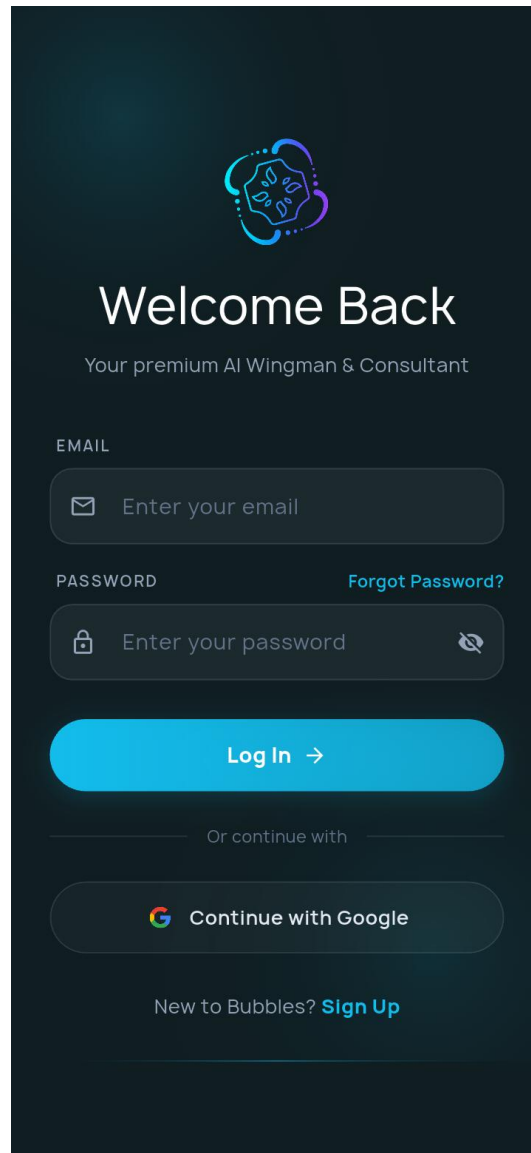


Figure 3: Login

The login screen authenticates returning users with their email and password. Credentials are verified against Supabase Auth, and a valid session token unlocks the rest of the application. A Google sign-in option and a password-reset link are also provided.

← Verify Email

Create Account

Join your personal Wingman today.

EMAIL

ma797509@gmail.com

PASSWORD

We have sent a verification link to your email. Tap the link in the email to continue.

CONFIRM PASSWORD

Sign Up

Reset Email

Continue with Google

Already have an account? [Log In](#)

Figure 4: Sign Up

New users create an account from the sign up screen by providing an email and a secure password. On submission the app provisions a Supabase identity and triggers an email verification step before onboarding begins.

← Profile Setup



Complete Profile

Tell us a bit more about yourself



FULL NAME

 Muhammad Ahmad

GENDER

 Male

DATE OF BIRTH

 2/1/2000

COUNTRY

 Pakistan

DATE OF BIRTH

 2/1/2000

COUNTRY

 Pakistan

Figure 5: Profile Completion

After verification the user completes a short profile with details such as display name, gender, country and date of birth. This information personalises the experience and seeds the user record referenced throughout the app.

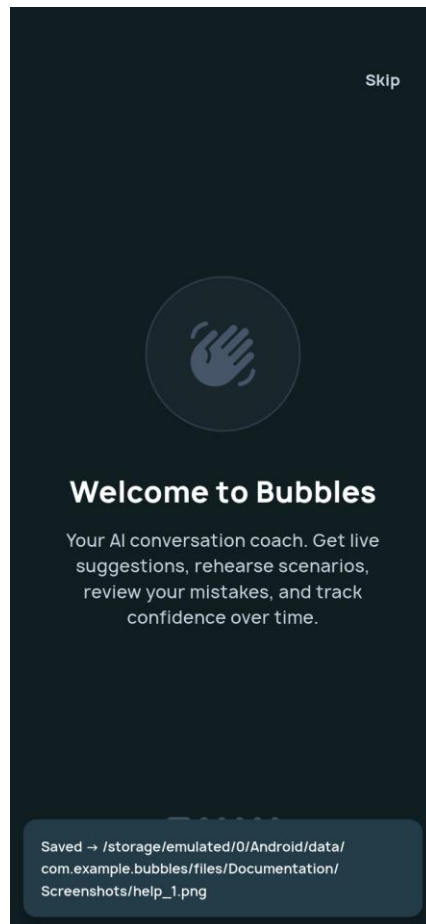


Figure 6: Onboarding

A guided walkthrough introduces the core value of Bubbles across a sequence of slides. It frames the real-time coaching, session review and progress features the user is about to configure.



Figure 7: Performa Wizard: Introduction

The Performa wizard captures the speaking profile that conditions every AI response. This introductory panel explains the purpose of the profile and previews the steps that follow.

About you
Step 1 of 3

Display name
Muhammad Ahmad

Age range
18-24

Primary role

Student Teacher Professional

Manager Freelancer Homemaker

Other

Profession detail
BSCS Student, 8th Smester

Expertise tags (max 5)

Python × Flutter ×

Add tag
Press enter to add +

Next →

Figure 8: Performa Wizard: Identity

The first step records identity context including display name, age range and primary role. These attributes let the assistant tailor its tone and vocabulary to the individual user.

< **Language & style**
Step 2 of 3

Native language
ur

Learning language
en

Proficiency (self-rated)

☐ beginner

☒ intermediate

☐ advanced

☐ fluent

Formality preference

☐ casual

☒ neutral

☐ formal

☐ mixed

Communication style (max 3)

assertive friendly concise

detailed humorous empathetic

← Back Next →

Figure 9: Performa Wizard: Language and Proficiency

The second step captures the native language, the language being practised and a self-rated proficiency. The server uses these fields to calibrate corrections and the difficulty of its feedback.

< **Goals & context**
Step 3 of 3

Primary goals (max 3)

confidence fluency

interview_prep work_meetings

academic social presentations

Typical scenarios (max 4)

lectures 1on1_tutoring

work_meetings casual_chat

interviews presentations

customer_service

Cultural context

From Okara, Punjab, Pakistan
punjabi Speaking.

Topics or words to avoid

← Back C

Figure 10: Performa Wizard: Goals and Context

The final step gathers learning goals, typical scenarios and cultural context. Together they shape the persona that drives Wingman advice and the generated roleplay scenarios.

Goals & context
Step 3 of 3

Primary goals (max 3)

- confidence
- fluency
- interview_prep
- work_meetings
- academic
- social
- presentations

Typical scenarios (max 4)

- lectures
- 1on1_tutoring
- work_meetings
- casual_chat
- interviews
- presentations
- customer_service

Cultural context

From Okara, Punjab, Pakistan
punjabi Speaking.

Topics or words to avoid

[Back](#) [Finish ✓](#)

Figure 11: Performa Wizard: Review

Before saving, the wizard summarises the captured profile for review. Confirming here persists the Performa to the backend and seeds the user's knowledge graph.

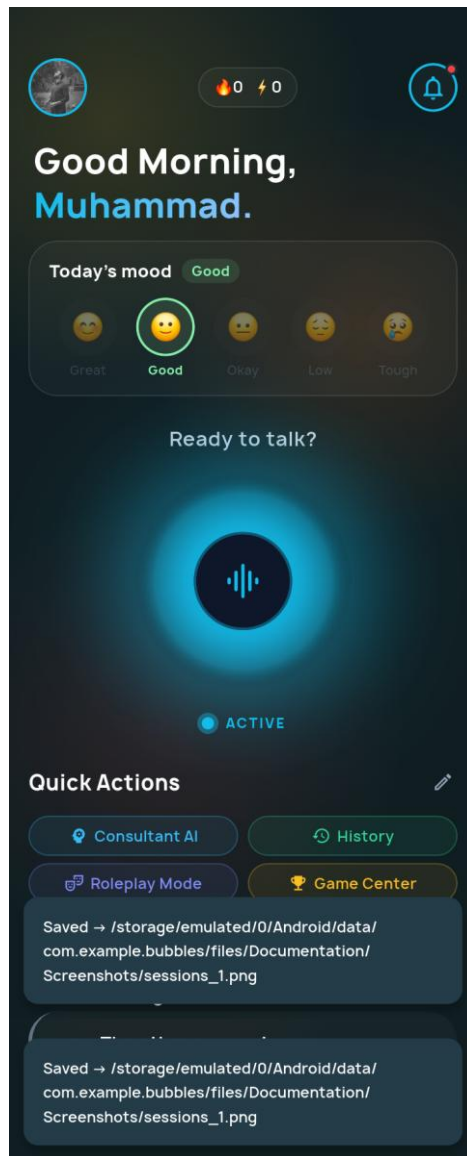


Figure 12: Home Screen (Listening State)

The home screen is the central hub after sign in. In its listening state the animated orb indicates that Bubbles is ready to capture a conversation, with quick actions and recent insights close at hand.

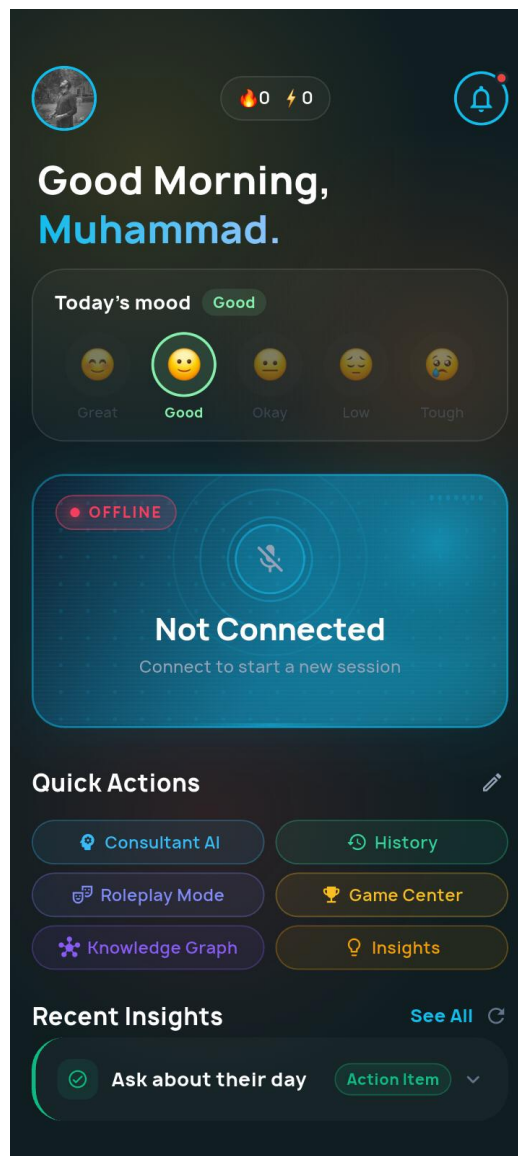


Figure 13: Home Screen (Server Not Connected)

When the backend is unreachable the home screen surfaces a clear Not Connected status. The user can still navigate the app while a banner signals that live features require a server connection.

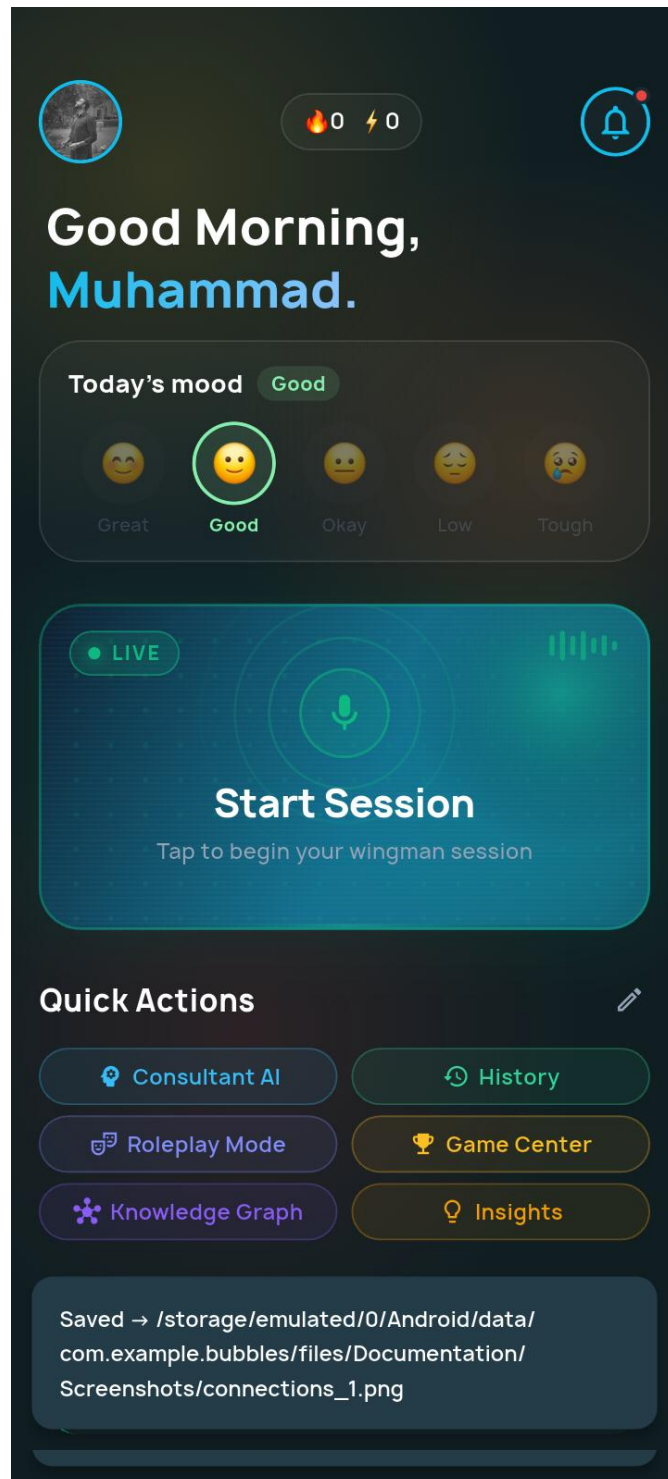


Figure 14: Home Screen (Server Connected)

With the server connected the home screen presents a prominent Start Session action. From here the user can immediately begin a live, coached conversation.

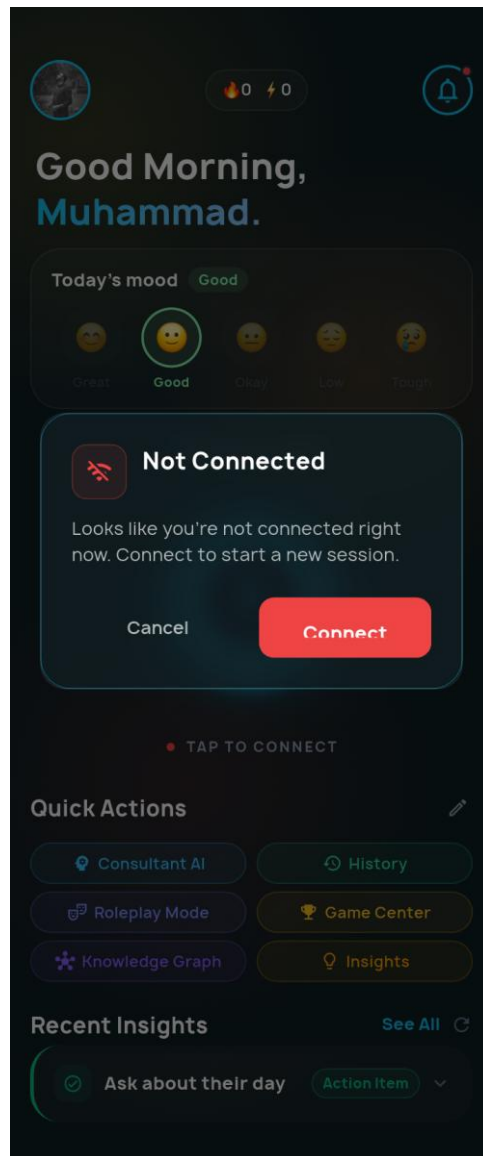


Figure 15: Home Screen: Connection Error

If a connection attempt fails, a dialog explains the problem and offers to retry. This keeps the user informed and provides a direct path to recover the session.



Figure 16: Entities Sheet

The entities sheet lists the people, organisations and topics extracted from the user's conversations. It provides quick access to the long-term memory that Bubbles maintains.

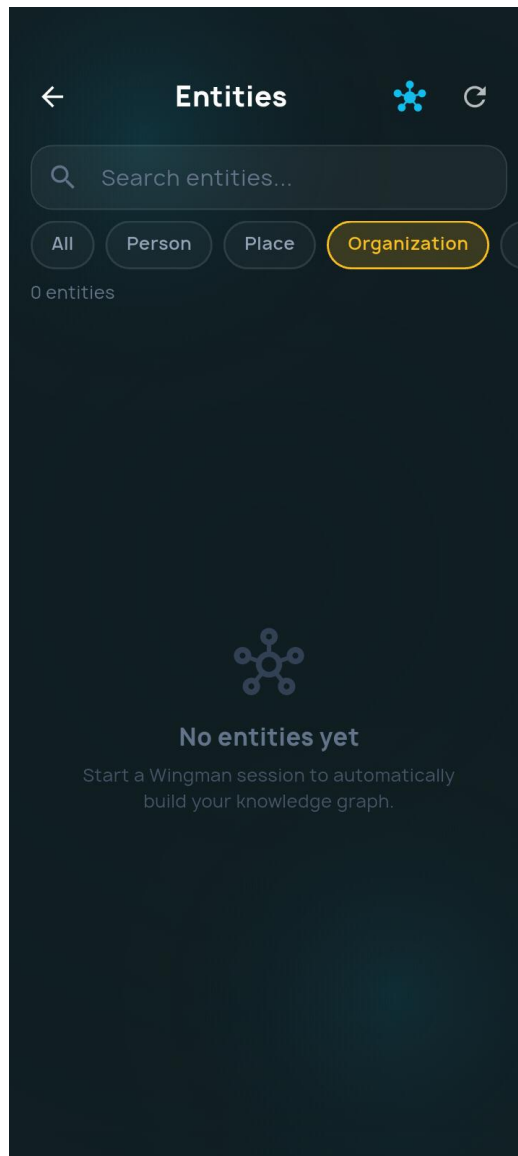


Figure 17: Entities Sheet (Empty State)

Before any sessions are recorded the entities sheet shows an empty state. It reassures new users that their knowledge graph will populate automatically as they use the app.

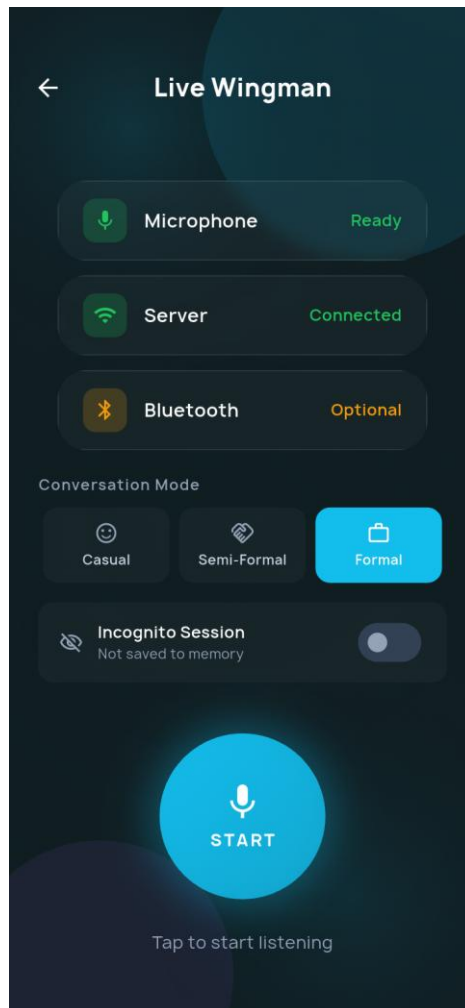


Figure 18: New Session Setup

The new session screen prepares a live conversation for real-time coaching. The user selects a scenario and grants microphone access before recording begins.

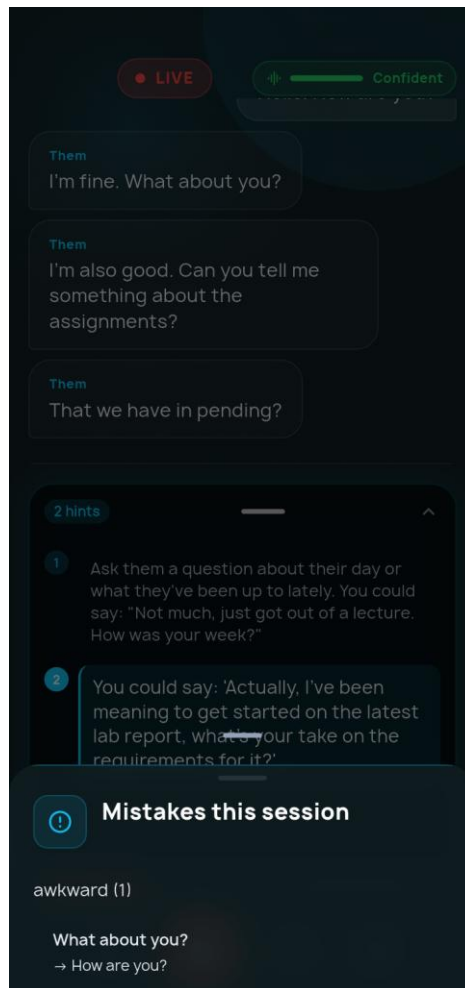


Figure 19: New Session: Add Context

An optional context sheet lets the user describe the meeting or scenario before recording. This context is attached to the session so the assistant can give more relevant advice.

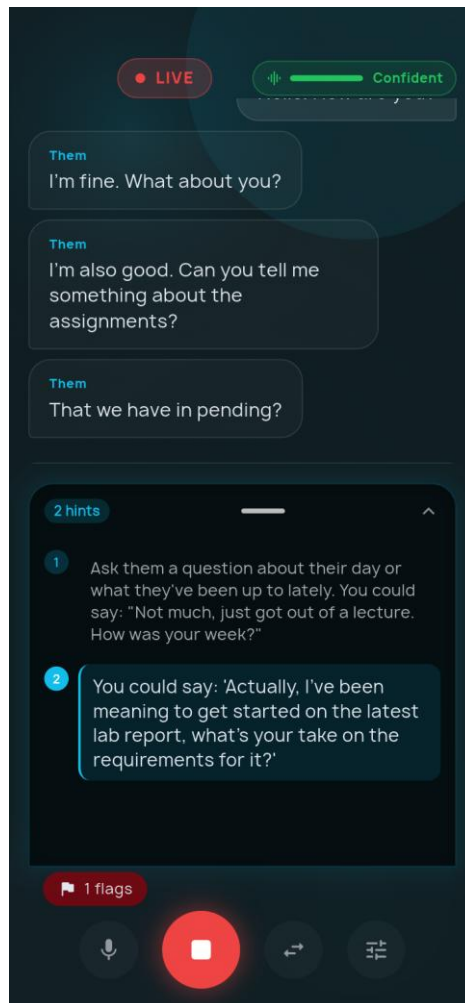


Figure 20: Live Session: Real-Time Transcription

During a live session Bubbles transcribes speech in real time and attributes each line to a speaker. The running transcript lets the user follow the conversation as it unfolds.

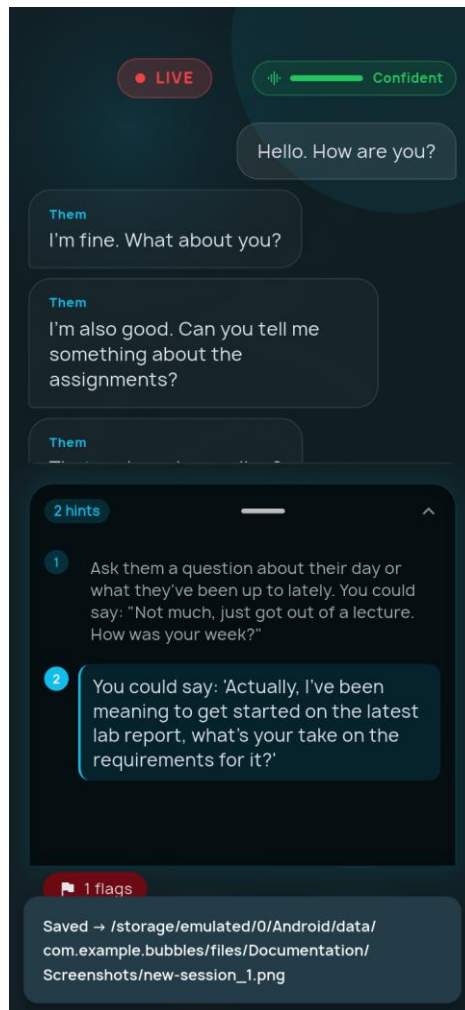


Figure 21: Live Session: Mistake Highlight

The assistant flags mistakes inline as the conversation progresses. Each highlight pinpoints the issue so the user can review and correct it after the session.

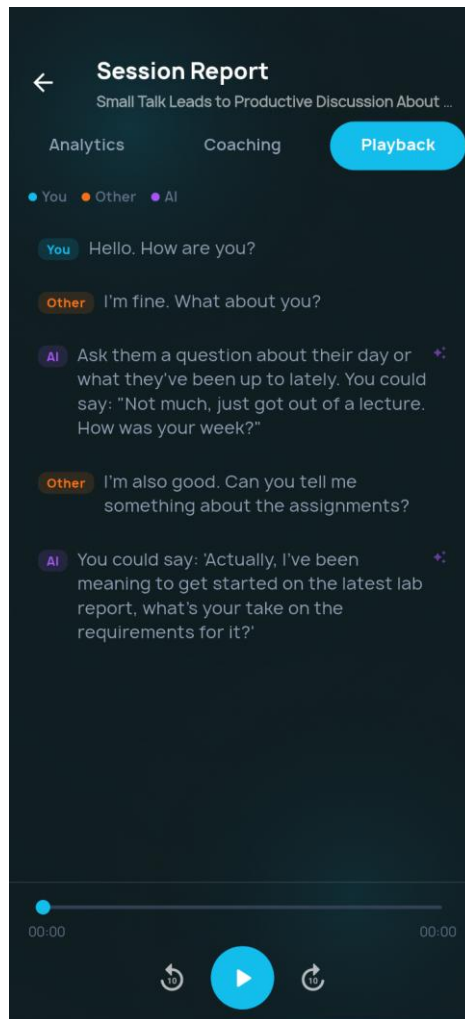


Figure 22: Live Session: Controls

The session controls let the user pause, resume or end a live conversation. They remain accessible throughout the session without obscuring the transcript.

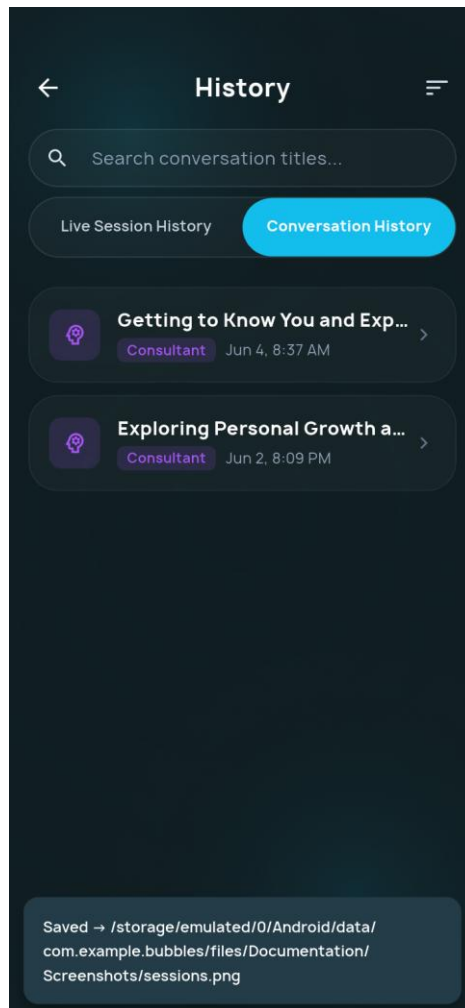


Figure 23: Session History

The history screen lists every past session with its key metadata at a glance. Tapping an entry opens the full report for that conversation.

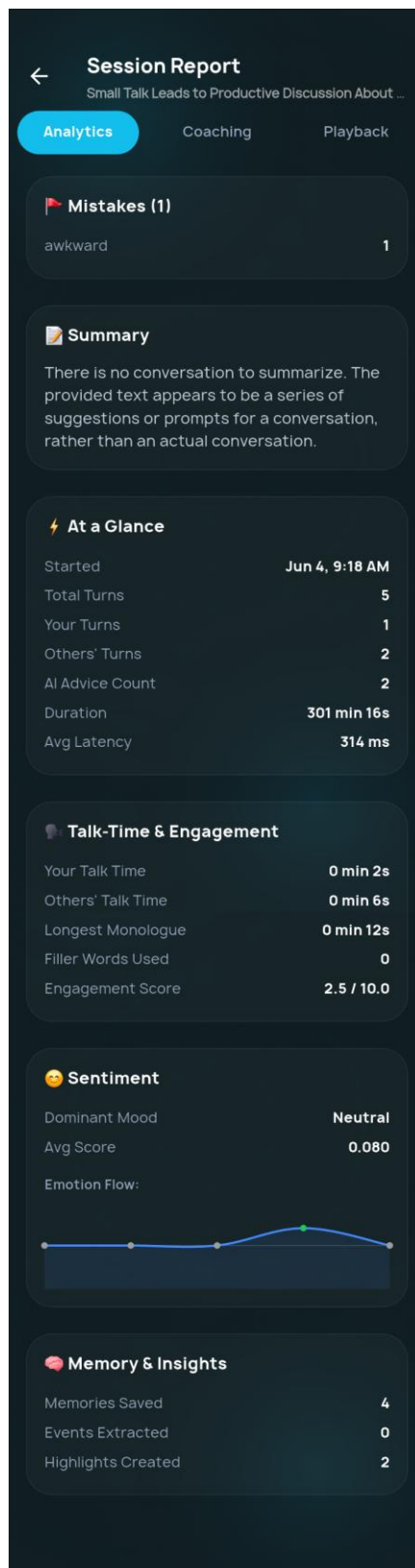


Figure 24: Session Report

The session report presents the complete transcript alongside extracted highlights and entities. It allows the user to revisit and learn from any previous conversation.

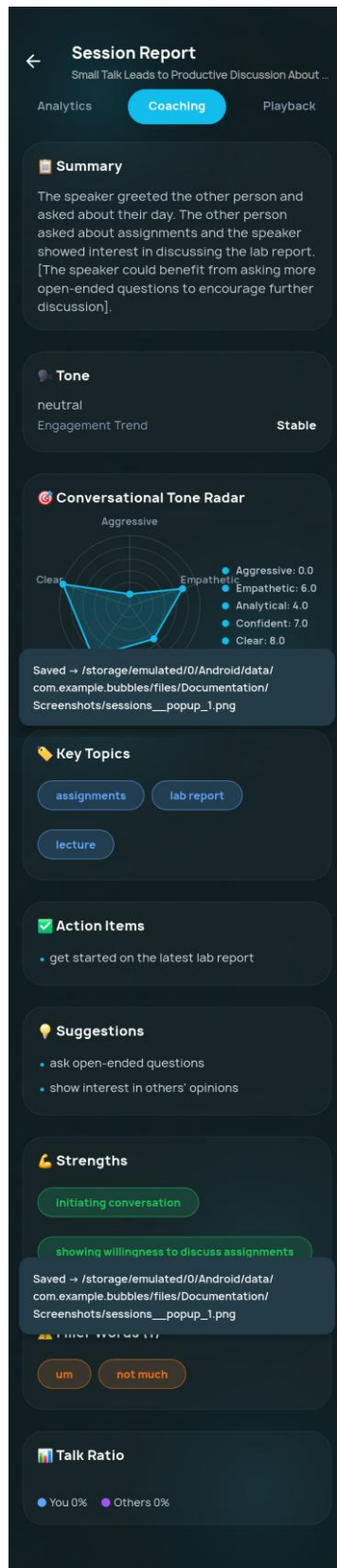


Figure 25: Session Report (Analytics)

Scrolling the report reveals per-session analytics such as fluency and confidence trends. These visualizations turn each conversation into actionable feedback.

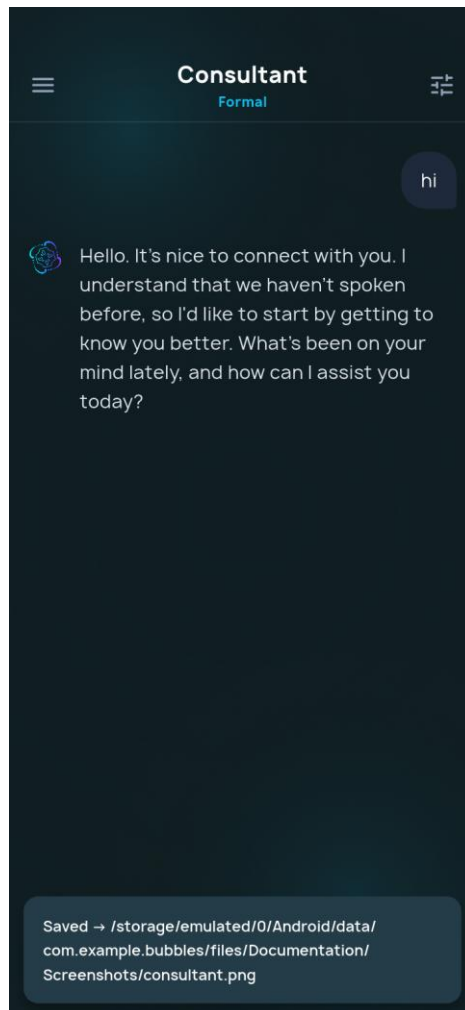


Figure 26: Consultant Chat

The consultant mode offers a conversational assistant for advice outside of a live session. Its responses are grounded in the user profile and the knowledge graph built from past sessions.

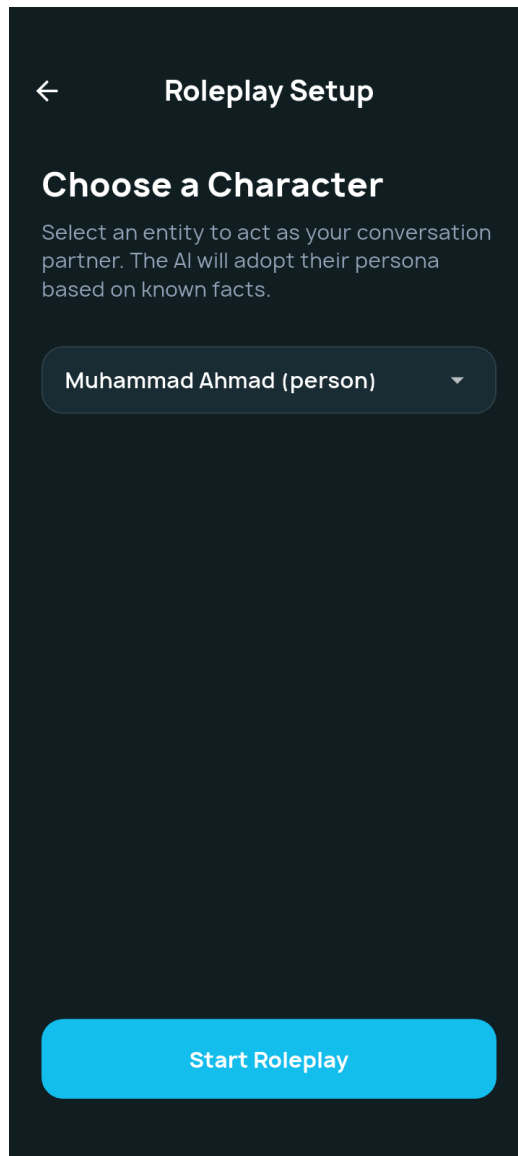


Figure 27: Roleplay Setup

Roleplay setup lets the user configure a simulated scenario and the role the assistant should play. The generated scenario adapts to the goals and proficiency captured in the Performa profile.

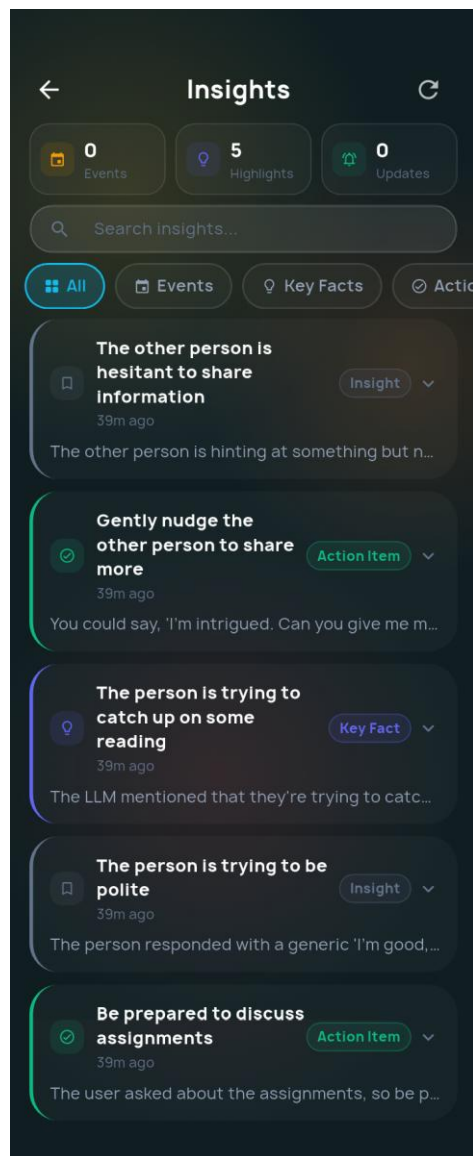


Figure 28: Insights

The insights screen aggregates performance across sessions into clear trends. Metrics such as fluency, confidence and recurring mistakes help the user track improvement over time.

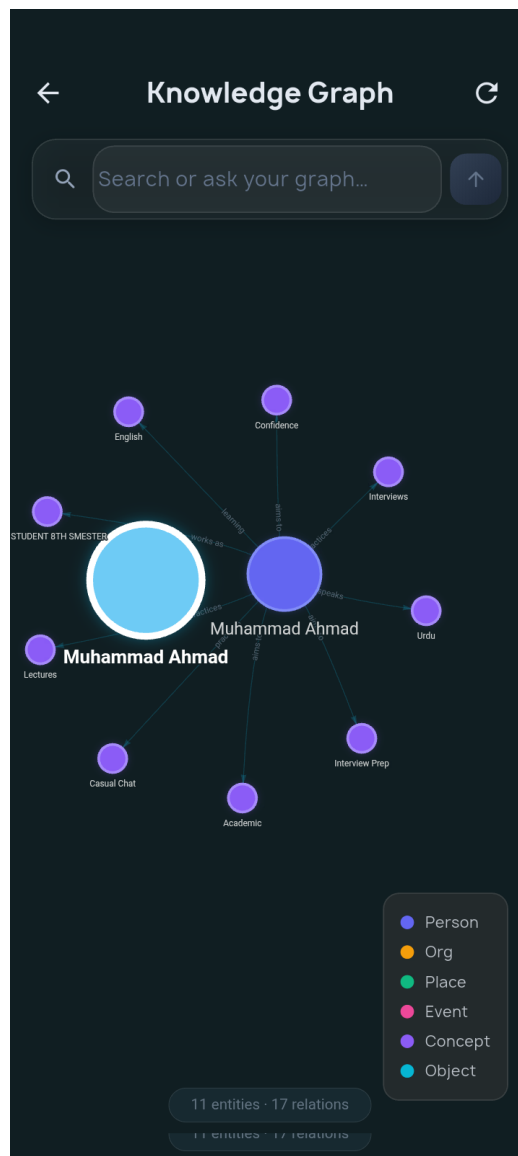


Figure 29: Knowledge Graph

The knowledge graph visualizes the people, topics and entities mentioned across sessions. It gives the assistant long-term memory and lets the user explore their conversational history.

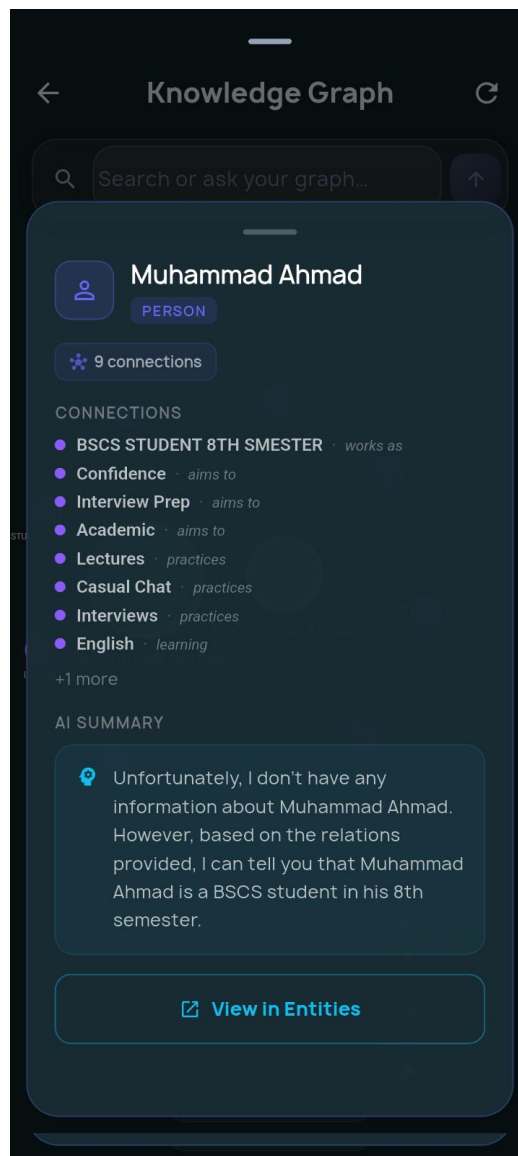


Figure 30: Knowledge Graph: Node Detail

Selecting a node reveals its details and the relationships that connect it to the rest of the graph. This makes it easy to trace how a person or topic recurs across conversations.

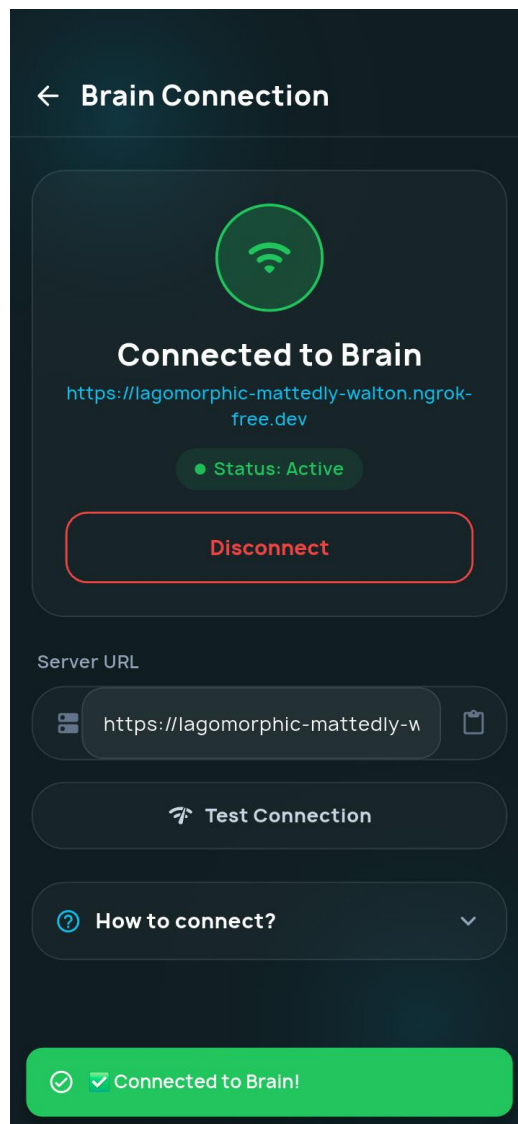


Figure 31: Connections

The connections screen focuses on the people the user interacts with most. Each contact is summarized from the cumulative context gathered across sessions.

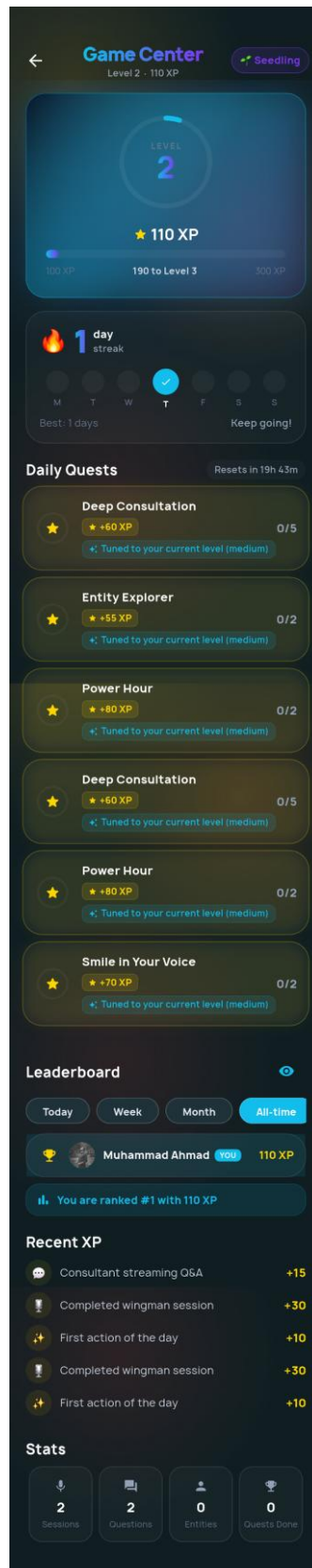


Figure 32: Game Center

The game center gamifies practice through quests, streaks and rewards. Daily challenges encourage consistent use and reinforce the habits that drive improvement.

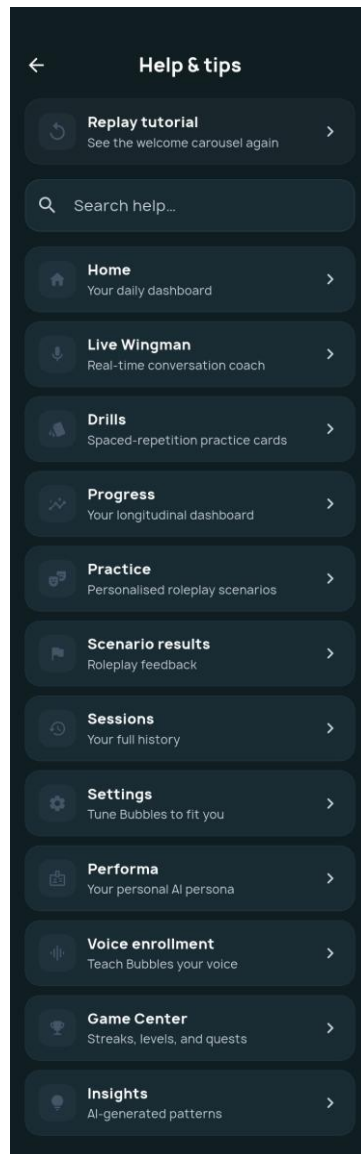


Figure 33: Help and Guide

The help screen documents each feature and answers common questions. It gives new users a reference for getting the most out of Bubbles.

The screenshot shows a mobile app interface for editing a 'Perfor...' profile. It features three main sections: 'Identity', 'Language & style', and 'Goals & context'. Each section contains a list of attributes and their values.

Identity	
Display name	Muhammad Ahmad
Age range	18-24
Primary role	Student
Profession	BSCS STUDENT 8TH SEMESTER
Expertise	—

Language & style	
Native language	ur
Learning language	en
Proficiency	intermediate
Formality	Neutral
Style	assertive, humorous, concise

Goals & context	
Primary goals	confidence, interview_prep, academic
Typical scenarios	lectures, casual_chat, interviews

Goals & context	
Primary goals	confidence, interview_prep, academic
Typical scenarios	lectures, casual_chat, interviews

Figure 34: Settings: Performa Profile

The Performa settings screen lets the user review and edit their speaking profile at any time. Saving changes refreshes the persona that the AI uses across the app.

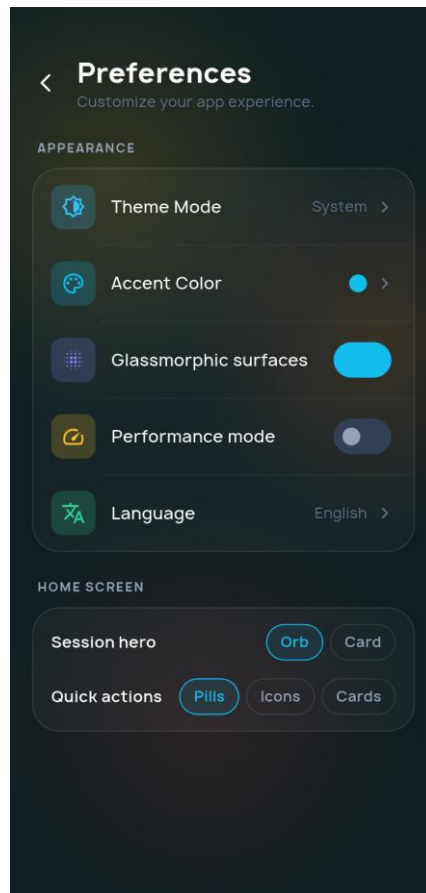


Figure 35: Settings: Preferences

Preferences control general app behavior such as theme, language and notifications. These options let the user tailor the interface to their liking.

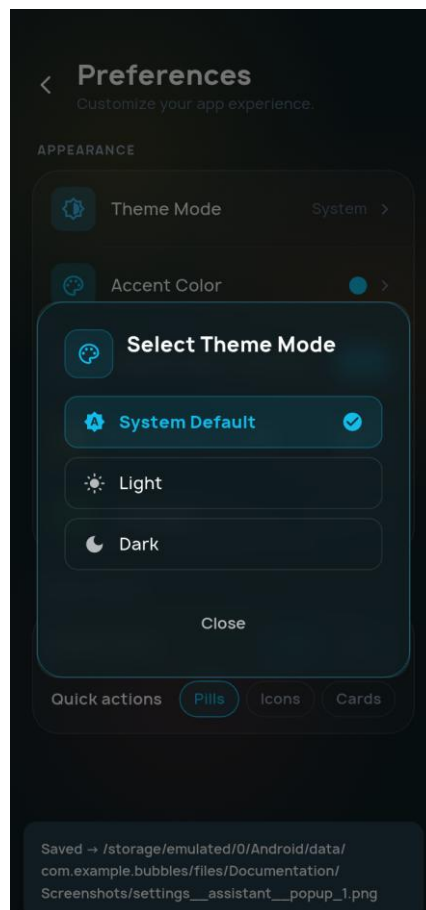


Figure 36: Settings: Preferences Picker

Selectable preferences open a focused picker dialog. The picker keeps choices such as language or theme clear and easy to change.

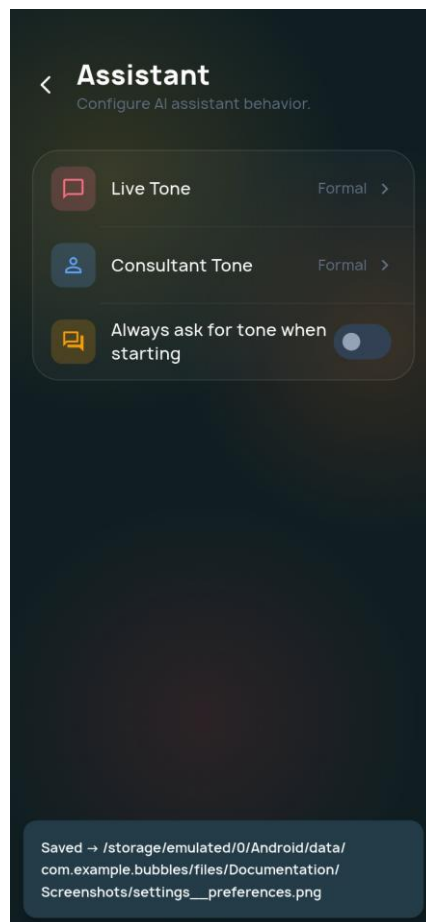


Figure 37: Settings: Assistant

Assistant settings expose how the AI behaves, including its advice style and verbosity. They give advanced users fine control over the coaching experience.

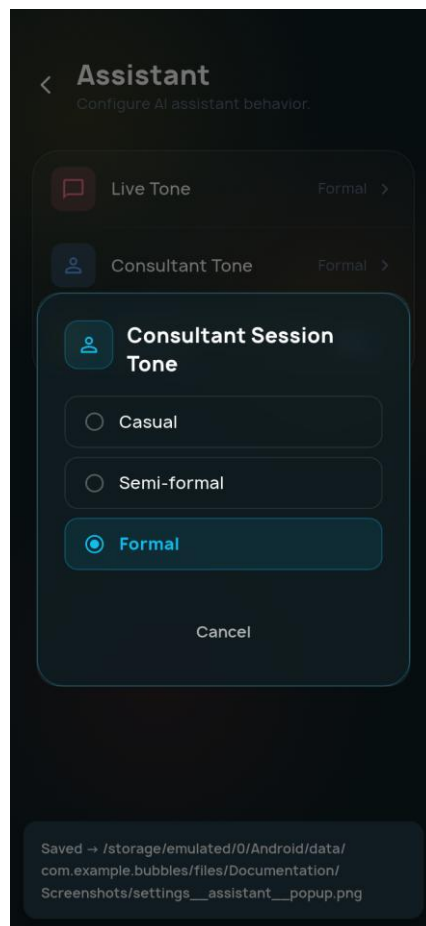


Figure 38: Settings: Assistant Picker

A picker dialog presents the available assistant options. It lets the user switch behaviors quickly without leaving the settings screen.

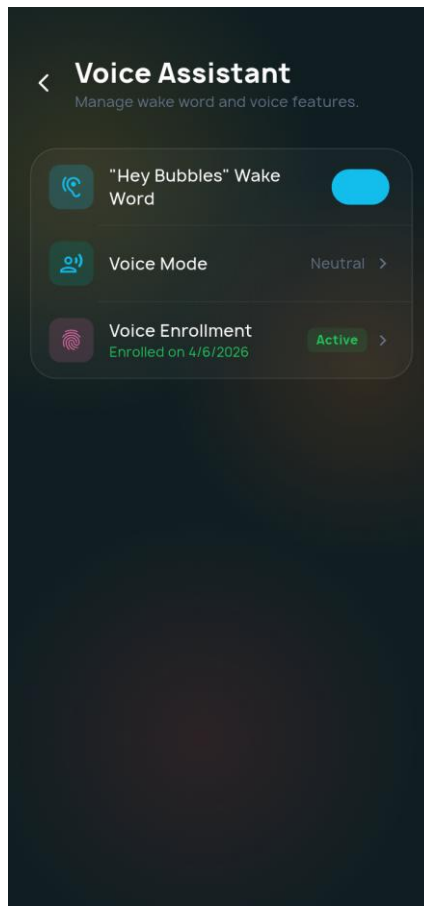


Figure 39: Settings: Voice Assistant

Voice assistant settings manage the wake word and hands-free voice controls. They configure how the user can invoke Bubbles simply by speaking.

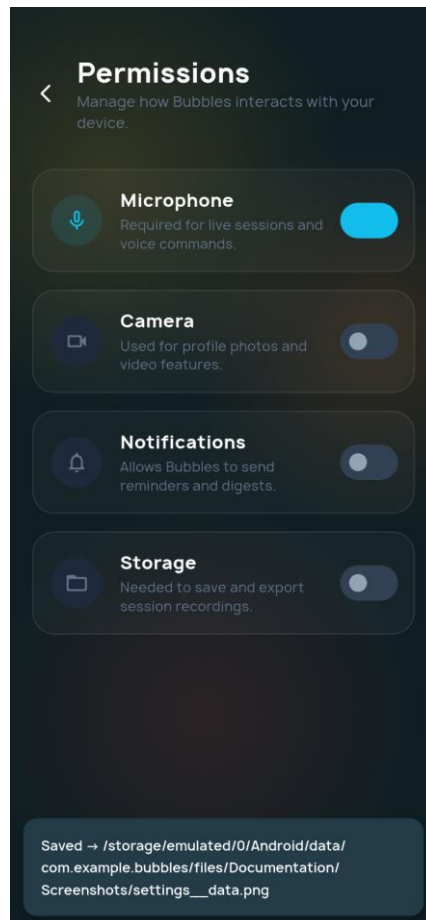


Figure 40: Settings: Permissions

The permissions screen summarizes the system permissions the app relies on, such as microphone access. It lets the user grant or revoke each permission from one place.

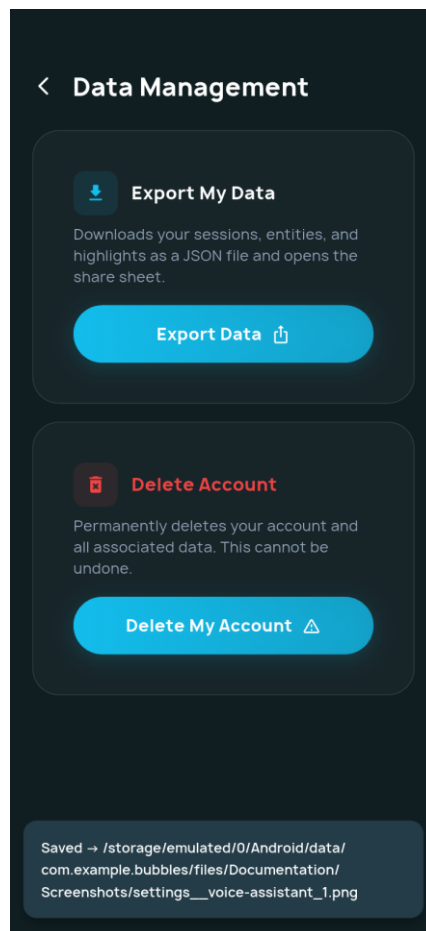


Figure 41: Settings: Data Management

Data management lets the user export their sessions, entities and highlights as JSON. The same screen provides account deletion in line with data privacy expectations.

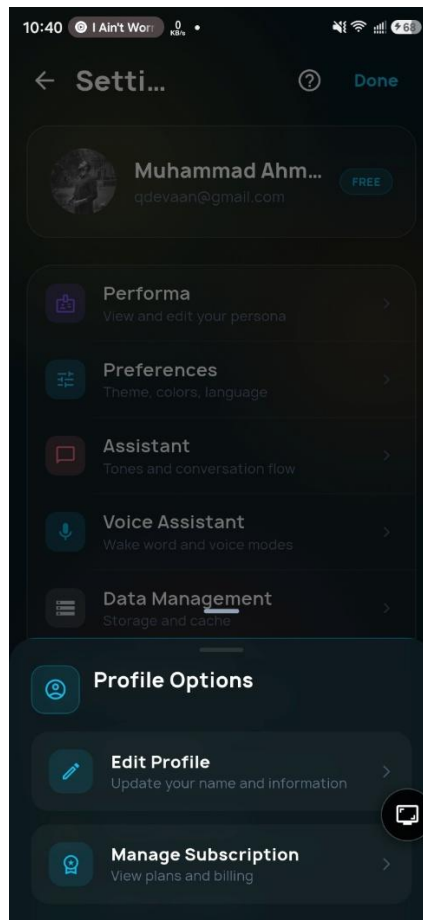


Figure 42: Settings: Profile Options

Profile options allow user to change the profile information they have saved in the app. They can also manage subscriptions if they have opt for one.

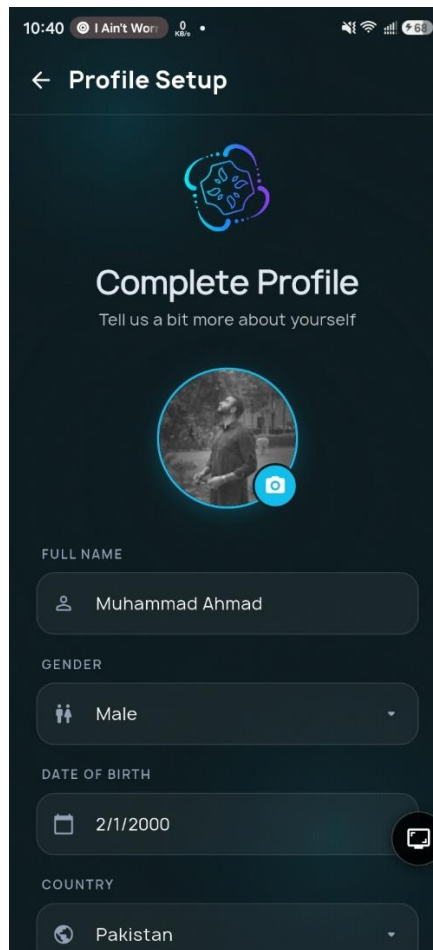


Figure 43: Settings: Edit Profile

This figure presents the application's edit Profile Setup interface, guiding users to complete their basic personal information. It features a profile picture upload option alongside standard input form fields for capturing essential details such as name, gender, date of birth, and country.

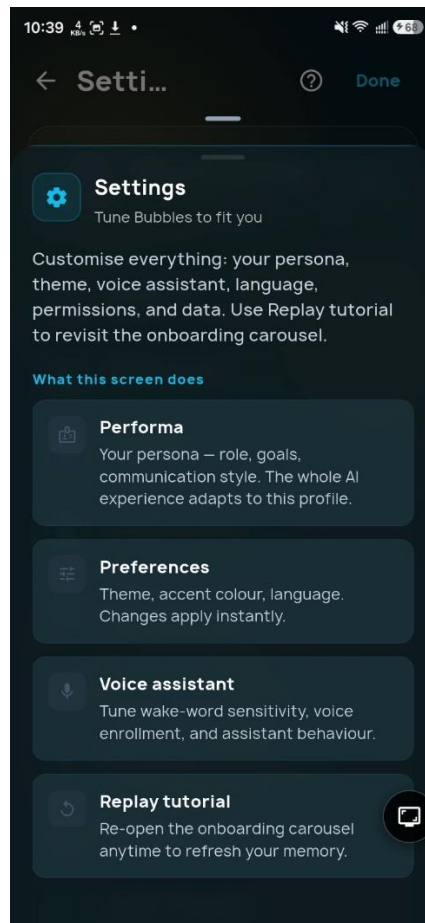


Figure 44: Settings: Help info_1

This figure displays the comprehensive Settings interface, serving as a central hub where users can customize their AI persona, visual preferences, and voice assistant behaviors. Additionally, the menu provides a helpful overview of each configuration category and offers quick access to replay the initial onboarding tutorial.

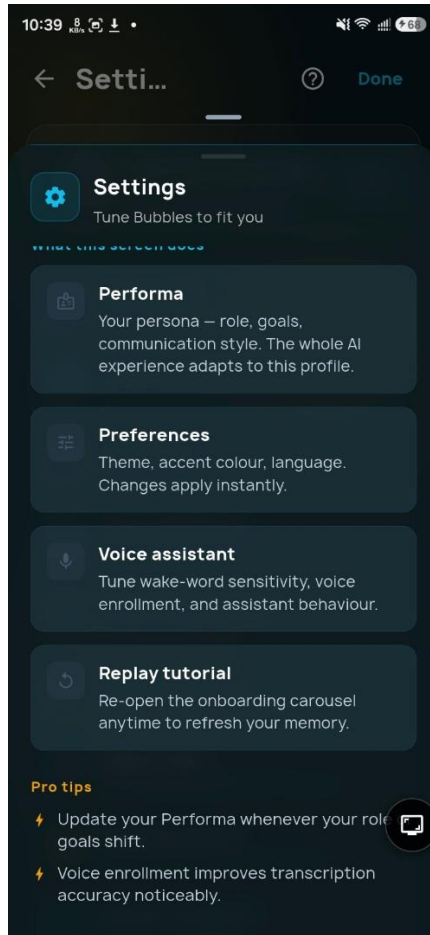


Figure 45: Settings: Help info_2

This figure presents the application's primary Settings menu, offering a centralized interface for users to personalize their overall experience. It features distinct navigational sections for configuring the AI "Performa" persona, adjusting visual preferences, tuning voice assistant behavior, and reviewing helpful pro tips.

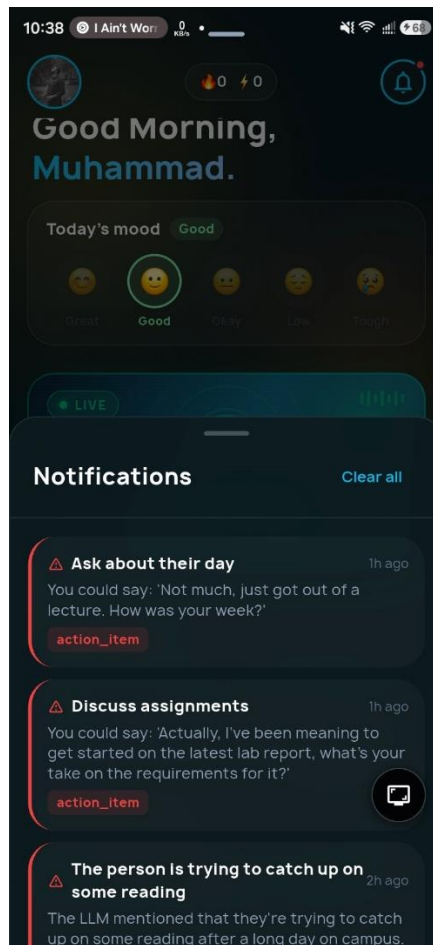


Figure 46: Notifications

This figure illustrates the application's home dashboard, featuring a personalized greeting and mood tracker partially obscured by an active Notifications overlay. The notifications panel provides AI-generated conversational action items and contextual insights to actively assist the user during interactions.

4.6 Challenges During Implementation

Developing a real-time AI application involving audio streaming and complex reasoning presented several significant technical hurdles:

4.6.1 Latency Management in Real-Time Advice

4.6.1.1 The Problem: The "Wingman" feature requires providing advice while the conversation is happening. Initially, chaining the processes (Speech-to-Text → Vector Search → LLM Generation → Response) caused delays of 3-5 seconds, which was too slow for natural conversation.

4.6.1.2 The Solution: Implemented a Parallel Execution Pipeline. The system was rearchitected to run the "Knowledge Extraction" and "Memory Saving" tasks in the background (fire-and-forget), while only the critical "Advice Generation" path was kept on the main thread. Additionally, switching to the faster Llama 3 8B model on Cerebras (fallback to Groq) hardware reduced inference time from ~2s to ~300ms.

4.6.2 Synchronizing Audio Streams & Speaker Identification

4.6.2.1 The Problem: Differentiating between the "User" and "Other" speakers in a single audio stream was difficult. Standard transcription often merged sentences from both speakers, confusing the AI about who said what.

4.6.2.2 The Solution: Integrated SpeechBrain ECAPA-TDNN for speaker diarization. We took users voice imprint, converted to embeddings which is the compared to Wingman mode audio to reliably attribute segments to the correct speaker, ensuring the Wingman only advises on what the other person said, not what the user themselves just said.

4.6.3 Hallucinations in the Consultant Node

4.6.3.1 The Problem: The LLM would occasionally invent facts when it didn't have enough context, or it would confuse similar past events.

4.6.3.2 The Solution: Adopted a Graph-Augmented Retrieval (GraphRAG) approach. Instead of relying solely on vector similarity (which is "fuzzy"), we forced the model to look up hard facts in the Knowledge Graph first. We also improved the "System Prompt" to explicitly instruct the model: "If you do not find the answer in the provided Context context, state that you do not know."

4.6.4 Managing State in a Stateless Environment

4.6.4.1 The Problem: HTTP servers (FastAPI) are inherently stateless, but a "Conversation" requires continuous state (graph nodes, recent message history). Losing state meant the AI "forgot" what it said 10 seconds ago.

4.6.4.2 The Solution: Implemented a Session Manager in the server's memory (LIVE_SESSIONS dictionary) and backed it up to Supabase. This allowed the

server to hold active conversation context in RAM for speed, while asynchronously persisting it to the database for reliability.

4.6.5 *Mobile Connectivity & Audio Quality*

4.6.5.1 *The Problem:* Streaming high-quality audio from a mobile device is unstable on cellular networks. Packet loss resulted in "choppy" audio that the AI could not transcribe.

4.6.5.2 *The Solution:* Created custom websocket for adaptive stream capability. We configured the client to automatically adjust bitrate based on network conditions and implemented a local buffer to ensure smooth transmission even during momentary connectivity drops.

Chapter 5. Results & Analysis

5.1 Experimental Setup

5.1.1 Testing Environment

The evaluation was conducted on dedicated servers on Hetzner under realistic operating conditions. It was hosted using 8 vCPU cores, 32 GB RAM, and running Ubuntu 22.04 LTS. Dedicated GPU was not required, since all heavy inference: transcription via Groq Whisper, and LLM generation via Cerebras Llama 3.1-8B was offloaded to external API providers. ECAPA-VoxCeleb speaker embedding model (192-dimensional embedding extraction, CPU-only) and the all-MiniLM-L6-v2 sentence transformer (384-dimensional embedding generation), both of which operated efficiently on CPU without exhausting memory.

The backend was containerized using Docker, running on Python 3.12 with FastAPI behind a ngrok reverse proxy, and created four uvicorn workers for concurrent request handling. Redis was employed for session state caching and ARQ job queue, while Supabase used as the primary database and vector store. The frontend was coded using Flutter (SDK 3.10+, Dart) running on Google pixel 8 pro and Samsung Galaxy S25 Ultra. Mobile microphone was used to capture voice and streamed to backend using websocket.

Monitoring and observability were done to capture latency and endpoint availability. Structured JSON logging via the python logging for all API requests, processing, and response events. OpenTelemetry provided distributed tracing across the FastAPI application, asyncpg database queries, and external HTTP calls. Latency measurements were captured using a dual-timestamp approach: the client recorded a send timestamp immediately before transmitting each audio chunk, and the server appended a receive timestamp on arrival.

5.1.2 Evaluation Metrics Per Subsystem

Each system inside Bubbles was tested using metrics chosen to reflect its functional requirements and quality attributes. Table 4 shows total set of metrics, operational definitions, and rationale.

Sub system	Metric	Why it matters
Groq Whisper (STT)	Word error rate	NFR02 requires $\geq 85\%$ accuracy in moderate noise
Groq Whisper (STT)	Transcription latency	Real-time coaching demands sub-second transcription
SpeechBrain diarisation	Identification Accuracy	Identity-aware coaching relies on reliably attributing speech to the correct participant
SpeechBrain diarisation	Voiceprint precision	Threshold tuning directly affects false acceptance and false rejection rates during enrolment matching
Wingman pipeline	End-to-end advice latency	NFR01 mandates suggestions within 2 seconds
Wingman pipeline	Advice relevance rate	Measures the practical coaching value delivered by the Wingman subsystem
Consultant RAG	Context relevance rate	Measures retrieval precision of the hybrid RAG engine
Consultant RAG	Response accuracy	Measures end-to-end answer quality of the Consultant subsystem
Gamification	User engagement	Measures motivational effectiveness of gamification

Table 4: Evaluation Metrics for Each Subsystem

5.1.3 Test dataset

The evaluation dataset was generated through live sessions conducted by us, following the procedure described in 3.4.4. A total of 20+ sessions were recorded, it included formal (structured meetings with defined agendas), semi-formal (brainstorming discussions and collaborative problem-solving), and casual (open-ended social chats). Duration for the sessions were, short (about 2 minutes), medium (up to 10 minutes) and long (over 20-30 minutes). The app was tested in quiet indoor rooms, medium noisy rooms and in public noisy place. Speaker configuration included user providing voice imprints and any other speaker in the room. All the sessions produced more than 3500 lines of transcribed text and about 700 entities and relations were extracted in knowledge graph.

5.2 Results

This section has quantitative and qualitative results for each of the subsystem in Bubbles. Results are organized by subsystem, using evaluation methods and metrics defined in section 5.1. All figures are percentiles compared to different trials and accuracy metrics calculation is based on ground truth where possible.

5.2.1 Transcription Performance (Groq Whisper)

The Groq Whisper model is used for Speech-to-text subsystem. The word error rate (WER) is computed by comparing the transcription of Groq to manually transcription for subset of sessions that included all the scenarios and noise. Table 5 shows WER against varying environments.

Condition	WER (%)	Notes
Quiet Room, 2 speakers	05 to 08	Clean audio, minimal overlap
Moderate noise, 2 speakers	10 to 15	Background chatter
Quiet room, 3+ speakers	08 to 12	Overlapping speech increases errors
Outdoor, 2+ speakers	15 to 20	Wind, traffic, and people noise

Table 5: Groq Whisper Transcription Accuracy

In quiet indoor environments, WER remained between 5% to 8% which exceeds our requirement of NFR02 of 85%. As we changed environment to a bit noiser and filled with ambient sounds WER shot up to 10% to 15% but well within the acceptable range. As we increased number of participants and add more noise, the overlapping of voices while simultaneously noise coming through degraded the results to 15% to 20% which is a known pattern for Groq Whisper models under these conditions. The latency for STT was below 2 seconds, a requirement of NFR01.

5.2.2 Speaker Identification (SpeechBrain ECAPA-VoxCeleb)

Speaker Identification was powered using SpeechBrain ECAPA-VoxCeleb model. A segment of audio was used to convert to 192-dimensional voiceprint embedding which was compared against user stored voiceprint in supabase. Using pgvectors cosine distance, a segment was labelled user if it fell below the threshold of 0.3, as required by FR03. Table 6 summarizes speaker identification metrics for all test sessions.

Metric	Value
Speaker identification accuracy (%)	88 to 94
User detection precision (%)	90 to 95
User detection recall (%)	85 to 92
Metric	Value

False positive rate (%)	03 to 08
Average voiceprint computation time (ms)	50 to 100

Table 6: Voiceprint-Based Speaker Identification at Threshold 0.30

The model achieved 88% to 94% accuracy over all sessions. The user detection precision was between 90% to 95% while recall was about 85% to 92%. False positive rate remained between 3% to 8% which shows that no-user being identified as user was very rare. Voice computation on CPU took about 50ms to 100ms per segment which is pretty negligible compared to scope of our project.

5.2.3 Wingman Response Latency

End to End latency for wingman was measured as the time from speaker speaking to advice being rendered on the mobile screen. We tested on 2 network options (wifi and mobile data) and two providers path (Cereberas Llama 3.1-8b as primary and Groq Llama 3.1-8B-instant as fallback). Table 7 shows p50, p95, p99, and maximum latency for each condition.

Network	Provider	p50 (ms)	p95 (ms)	p99 (ms)	Max (ms)
Wi-Fi	Cereberas	300 to 400	500 to 600	700 to 800	~1000
Wi-Fi	Groq	400 to 500	600 to 800	900 to 1000	~1200
Data	Cereberas	400 to 500	600 to 700	800 to 900	~1100
Data	Groq	500 to 600	700 to 900	1000 to 1200	~1500

Table 7: Wingman End-to-End Latency by Network Condition and Provider

Primary provider over Wi-Fi achieved a p50 latency of 300ms to 400ms under the target of NFR01 (≤ 2 seconds). The p95 remained under 600ms and p99 did just below 800ms. On the fallback provider there was additional cost of 100ms on each percentile due to slower LPU computation. When we switch to (mobile) data it adds an additional 100ms due to extra network round trip.

5.2.4 Hybrid RAG Retrieval Performance

The system heavily relies on hybrid RAG engine to get context for answering user questions. It combines vector-based semantic search (pgvectors cosine similarity on all-MiniLM-L6-v2 384 dimensional embeddings) with Knowledge Graph traversal (neighbourhood nodes searching). A manual testing was done on 25 random queries to see if the context fetched was relevant to the query. Table 8 summarises the results.

Retrieval Method	Context Relevance Rate (%)	Avg Retrieval Time (ms)
Vector-only (pgvector cosine similarity)	70 to 75	50 to 100
Graph-only (KG neighbourhood traversal)	65 to 70	80 to 150
Hybrid (vector + KG)	~95	100 to 120

Table 8: Hybrid RAG Retrieval Study

Vector only search achieved 70% to 75% relevancy, it excelled at finding past sessions where similar words were found but failed to find relational connections to that concept. On the other hand Graph only retrieval demonstrated opposite strength at relevance rate of 65% to 70%, it retrieved accurate factual relations but failed to find data in unlinked sessions. The hybrid

approach which combined both techniques, showed a great context relevance retrieval of approximately 95%, which perfectly satisfies our requirement of FR07.

5.2.6 End-to-End Pipeline Summary

Table 9 concludes the key results found against the non-functional requirements and design targets that were mentioned in Chapter 2 and 3. Every goal was either exceeded or met based on the requirements.

Requirement	Target	Achieved
NFR01: Latency $\leq 2s$	$\leq 2000ms$	Wingman: $\sim 400ms$ p50, $\sim 800ms$ p99
NFR02: STT Accuracy $\geq 85\%$	$\geq 85\%$ accuracy	$\sim 92\%$ in quiet, $\sim 85\text{--}90\%$ in moderate noise
RAG Contextual Awareness	Contextual answers	$\sim 95\%$ (Hybrid RAG)
Wingman Advice Latency	$< 500ms$	$\sim 400ms$ p50 on WiFi, Cerebras

Table 9: Consolidated Results vs Requirements

Table 9:

For over 20+ test session, the system generated about 3500 lines of transcribed text and 700 entities and relations. The Bubbles pipelines demonstrated promising results that it can deliver coaching within milliseconds, accurately identify speakers, and answer questions with greater and relevant context. The system meets measured criteria which are written in chapter 3 and 4, proving that system meets all design targets while maintaining natural flow of conversation in everyday use.

5.3 Comparative Analysis with Existing Work

This section provided comparison of Bubbles with application that already exists in the market, and that users use actively every day. The comparison is done based on core features that contributes to bubbles architecture. Table 5.3-1 presents the comparison of each of the feature.

5.3.1 Comparative Analysis Table

Feature Dimension	Otter.ai	Fathom	Wispr Flow	Yoodli	Google Assistant	Bubbles (Ours)
Primary function	Meeting notes and transcription	Meeting summaries	Dictation	Speech coaching	General assistant	Communication coaching
Target user	Teams	Teams	Individual	Individual	Everyone	Individual
Platform	Web, Mobile	Web, Mobile	PC, Mobile	Web	Mobile, Smart speaker	Mobile
Real-time coaching	✗ No (live transcript only)	✗ No (live summaries only)	✗ No (dictation only)	⚠ Partial (behavioural nudges)	✗ No (stateless replies)	✓ Yes (Wingman)
Identity-aware diarization	✗ Speaker 0/1	✗ Speaker 0/1	✗ Single speaker	✗ Single speaker	✗ N/A	✓ Yes (voiceprint, threshold 0.30)
Cross-session memory	✗ Per-meeting	✗ Per-meeting	✗ No	✗ Per-session metrics	⚠ Partial (LLM past-chat recall)	✓ Yes (KG RAG + pyvector)
Retrieval method	Basic keyword search	Basic keyword search	None	None	LLM context window	Hybrid (vector + relational KG)
Ganify	✗ No	✗ No	✗ No	✗ No	✗ No	✓ Yes (XP, levels, quests)
Model architecture	Single ASR model	Single ASR model	Single ASR model	Single ASR, classifier model	Single LLM	Multi-model pipeline (6+ models)
Privacy model	Enterprise data harvesting	Enterprise data harvesting	Cloud processing	Cloud processing	Google ecosystem	Open-source, user-controlled

Table 10: Comparative Analysis of Bubbles Against Existing Tools

5.3.2 Primary function

Otter.ai and Fathom are enterprise level software used by teams, they extract notes and generate summary at the end of a meeting. Wispr flow is a dictation for individuals on all platforms, essentially help them dictate their speech inside of writing themselves. Yoodli is speech coaching app that gives tips and nudges for interviews and public speaking. Google assistant is simply voice interface for queries and device controls. Bubbles, in contrast to all these is built for real-time coaching and improving users communication skills with ability to relearn from past sessions.

5.3.3 Target user

Otter and Fathom target teams and organisations for professional meetings and discussions. Their integrations of calendar and workspaces reflect enterprise priorities. Wispr Flow and Yoodli cater to individual needs but their use case is narrow. Bubbles intentionally target single user with personalised coaching; the identity-aware diarization, persistent memory, and gamification is optimised for one person's growth and improvement.

5.3.4 Real-time coaching

Otter and Fathom only provide post meeting summaries and key notes, no real-time feedback. Wispr flow only dictates the words with no layer of suggestion for user. Yoodli provides live nudges like slow down or reduce filler words, but these are behavioural recommendations, it doesn't analyse context of meeting. Google assistant only carry out actions asked by user, completely stateless. Bubbles fill this gap by providing context-aware suggestions to help user recall important points and provide feedback.

5.3.5 Identity-aware diarization:

Every existing solution label speakers anonymously. Otter and Fathom assigns "Speaker 0" and "Speaker 1". Wispr flow and Yoodli only work on single stream of audio, they have no speaker diarization. Bubbles on the other side ensures perfect speaker recognition. It uses user's voiceprint to compare with incoming audio and compare using cosine distance and based on threshold it assigns the chunk of audio to either user or other speakers.

5.3.6 Cross-session memory

Otter and Fathom are completely stateless, they only have memory of current meeting in which they participate. Yoodli tracks per-session metrics (filler word frequency, pacing trends) on a dashboard but does not retrieve past conversation content. Google's Gemini has introduced new feature in 2025 that allows it to recall past conversation and use it as context, however this is only saved as a memory it doesn't query past sessions to retrieve information. Bubbles' hybrid RAG allows it to have unlimited memory, where it can retrieve past sessions, look into different entities and their relations.

5.3.7 Retrieval method

Otter and Fathom provide basic keyword search on transcribed text, which is limited to only string search or near-string search. Wispr Flow and Yoodli does not provide any mechanism of searching through past conversations. Google Gemini can recall things from its memory but it doesn't have a proper query technique to search through past sessions and is limited by the LLM's context window. Bubbles as discussed earlier uses hybrid RAG as its retrieval method. It makes use of its constantly expanding Knowledge graph of entities and relationships. Furthermore, it uses vector embedding to do cosine similarity on user query and past sessions, for which it uses

all-MiniLM-L6-v2 384-dimensional. Lastly it combines both sources to create context for LLM to answer user query.

5.3.8 Gamification

Otter, Fathom, Wispr Flow, Yoodli, and Google Assistant have no gamification layer. Bubbles on the other hand has taken inspiration from Duolingo, which is largely successful due to its gamification system that keeps users hooked. User can earn XP for completing sessions, daily quests and challenges. This layer fills the motivation gap for the users.

5.3.9 Model architecture

All competitors make use of only single-model or single-task architecture. Otter and fathom use only one model for speech-to-text, with limited NLP for summary generation. Wispr uses single model for dictating speech and Google Gemini is single LLM used to complete user queries. Bubbles orchestrates six models: Groq Whisper (STT), SpeechBrain ECAPA-VoxCeleb (diarisation), all-MiniLM-L6-v2 (embeddings), a fast 8B-parameter LLM (Wingman), a deep-reasoning 70B-parameter LLM (Consultant), and a rule-based gamification engine. Each subsystem is assigned a model based on its uses case.

5.3.10 Privacy model.

Otter and Fathom are enterprise software which use the users data for analytics. Google process its queries on its servers and show you relevant ads. On contrary to all this Bubbles is Open-source for anyone to fork and use, in way which best fit their usecase.

5.4 Discussion on Findings

5.4.1 Multi-Model Architecture: The Right Tool for Each Task

A single model approach cannot satisfy the demands of real-time coaching. ASR needs sub-second latency, diarization needs proper speaker identification, real-time advice needs fast inference and deep questions require larger reasoning models. This forces Bubbles to equip multi-model architecture: Wingman uses Ceberas for speed, Consultant needs Groq for depth, diarization require SpeechBrian. Platforms like Otter and Yoodli use single model that does not fill the requirement of Bubbles.

5.4.2 Hybrid RAG: Why Vector Search Alone Was Insufficient

Vector similarity search only accounted for 70-75% relevance. But it could not answer relation queries, like “What did I promise Sarah?”, vector search found the text with word promise and Sarah and similar independently, but missed structural connection between both entities. The hybrid approach simultaneously found sessions using vector search and information regarding entities and their relations, merging both gave us relevancy of about ~95%.

5.4.3 Identity-Aware Diarization: From Anonymous Labels to Named Speakers

Standard diarization label speakers anonymously like speakr 0/1. This is insufficient as it can not find which person spoke. Bubbles added a verification layer using SpeechBrain ECAPA-VoxCeleb. It uses 192-dim voiceprint per segment, compared to user voiceprint for identifying their segment. This enables wingman mode to be able to give suggestions based on what the other person said instead of getting confused between anonymous speakers. The onlys limitation is binary classification but since Bubbles is centred around single user this works fine for our case.

5.4.5 Latency Optimization: From 5 Seconds to 500 Milliseconds

The initial serial pipeline of audio, STT, vector search, Knowledge graph traversal, creating new entities and relations, LLM response generation, and network hops took 3-5 seconds for real-time conversation. We optimised this by using parallel programming. Session storage and context-update was designed to background tasks. The hybrid RAG was parallelised by running vector search and graph traversal in background and parallel as the user speak. The main thread only had 1 simple pipeline: audio, STT, LLM and display on screen. We also switched to light models for wingman and larger for consultant mode. The key point was parallelising tasks where possible and using different providers and models to decrease the latency.

5.4.6 Limitations and Threats to Validity

WER degrades to 15% to 20% in noisy background, posing a huge problem for everyday use. Speaker identification threshold was calibrated by us based on our trial sessions of small number of people. The 20+ test sessions is a small number of samples which may have some bias. The system depends on stable internet connection, performance under poor network was not tested. These limitations show the results are upper-bound estimates

Chapter 6. Conclusions

6.1 Summary

6.1.1 Project Overview

Bubbles is an AI-powered mobile application that assists user in their day-to-day interactions. Unlike existing competitors in market that are stateless and team-centric, Bubbles presents itself as individual-first solution. It acts as a personal coach and assistant by tracking conversations, analysing sessions and building long-term memory of user's interactions.

6.1.2 Core features

Live wingman mode: Captures audio to provide real-time transcription, identify speakers, and show instant suggestion on screen without interrupting the flow of conversation.

Consultant Mode: An indepth Q&A mode where user can ask questions about past sessions in greater detail.

Roleplay Mode: Enables users to converse with entities extracted from sessions to rehearse conversations with them.

Gamification: Uses a Duolingo-inspired gamification system of XP, levels, daily quests, and challenges, and streaks to keep users motivated to improve their communication skills.

Insights Dashboard: Shows events, key facts, and action items extracted from conversations so users can review them later, see different conversations and ask questions about each entity.

6.1.3 Technical Architecture

Frontend: Developed using Flutter to create a cross-platform mobile application.

Backend: Built with Python and FastAPI and custom WebSockets for real-time, low-latency audio streaming.

AI & Inference:

- Speech to text: Uses Groq Whisper (large-v3-turbo) for fast transcription.
- Speaker Diarization: Uses SpeechBrain (ECAPA-TDNN) to identify the user by computing cosine distance using 192-dimensional voiceprint.
- LLMs: Uses a multi-provider architecture; Llama 3 (via Cerebras and Groq) for real-time advice and deep reasoning.

Data & Memory (Hybrid RAG): A merger of vector similarity search on user queries and sessions with Knowledge Graph traversal, to retrieve accurate and factual context.

6.1.4 Results & Evaluation

The Bubbles was evaluated on more than 30 test sessions, showing successful results:

- Latency: Wingman mode suggested feedback within 400ms easily meeting the initial ≤ 2 second requirement.
- Accuracy: Audio transcription accuracy touched ~92% in quiet environments and was about 85% to 90% in moderate noise.
- Contextual Retrieval: The Hybrid RAG (combining vectors and the Knowledge Graph) gave a ~95% context relevant answer to queries.

6.2 Recommendations for Future Work

While Bubbles successfully demonstrated its capability of real-time advice during conversation, key insights after the conversation and context-aware coaching assistance there is still room for improvement. Based on the system's goals, constraints and evaluation results, there are areas where it can be further improved and optimized.

6.2.1 Edge Computing and Offline Execution

The current architecture heavily depends on stable internet connections to use the backend and models. Future version of the Bubbles should include deploying on-device models that are quantized to fit within the mobile devices commonly used. Lightweight LLM like Gemma and ASR module like tiny-whisper can be equipped for this too. Attempting this would enable offline functionality, full privacy and independence of network induced latency.

6.2.2 Enhanced Noise Suppression

Trials sessions show that Word Error Rate (WER) can go upto 15% to 20% in noisy background, which poses a huge problem for everyday use. Future models should research different algorithms for better noise suppression, and isolate user's voice from background noise.

6.2.3 Multi-speaker Diarization

Currently, Bubbles only support binary speaker diarization which is not ideal for common use. We propose research for advanced diarization models that can easily segment multiple users in an conversation, which the user can letter tag with their names for better context formation (entities and relations).

6.2.4 Multilingual Expansion

At present, the application support transcription and contextual reasoning for English language, but in Pakistan Urdu is spoken widely. LLM have multi-lingual abilities, but require fine-tuning for greater context extraction and retrieval. Along with this there is a need for either new STT model for urdu transcription or whisper variant can be fine tuned to transcribe urdu audio. Later other languages can be incorporated into the system, which will help broaden the accessibility and real-world applicability.

6.2.4 Multimodal integration (Video and Facial recognition):

Current scope of Bubbles involves only audio as the only source of information for context and coaching. Intenerating video analysis and facial recognition will further improve its core features. Video can help with diarization by recording lip movement, and it will assist in better coaching by capturing facial expressions and body language. This will allow for more in-depth sentiment analysis and coaching feedback.

Chapter 7. References

- [1] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, “*X-vectors: Robust DNN Embeddings for Speaker Recognition*,” in **Proc. IEEE ICASSP**, 2018.
- [2] D. Snyder, D. Garcia-Romero, G. Sell, A. McCree, D. Povey, and S. Khudanpur, “*Speaker Recognition for Multi-Speaker Conversations Using X-Vectors*,” in **Proc. IEEE ICASSP**, 2019.
- [3] F. Landini, J. Profant, M. Diez, and L. Burget, “*Bayesian HMM clustering of x-vector sequences (VBx) in speaker diarization: theory, implementation and analysis on standard tasks*,” **Computer Speech & Language**, vol. 71, 2022.
- [4] Y. Fujita, N. Kanda, S. Horiguchi, K. Nagamatsu, and S. Watanabe, “*End-to-End Neural Speaker Diarization with Permutation-Free Objectives*,” in **Proc. Interspeech**, 2019.
- [5] S. Horiguchi, Y. Fujita, S. Watanabe, Y. Xu, and K. Nagamatsu, “*End-to-End Speaker Diarization for an Unknown Number of Speakers with Encoder-Decoder Based Attractors*,” in **Proc. Interspeech**, 2020.
- [6] I. Medennikov *et al.*, “*Target-Speaker Voice Activity Detection: a Novel Approach for Multi-Speaker Diarization in a Dinner Party Scenario*,” in **Proc. Interspeech**, 2020.
- [7] H. Bredin *et al.*, “*Pyannote.audio: Neural building blocks for speaker diarization*,” in **Proc. IEEE ICASSP**, 2020.
- [8] A. Radford *et al.*, “*Robust Speech Recognition via Large-Scale Weak Supervision*,” in **Proc. ICML**, 2023.
- [9] A. Baevski, Y. Zhou, A.-r. Mohamed, and M. Auli, “*wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*,” in **Advances in Neural Inf. Proc. Sys. (NeurIPS)**, 2020.
- [10] A. Gulati *et al.*, “*Conformer: Convolution-augmented Transformer for Speech Recognition*,” in **Proc. Interspeech**, 2020.
- [11] M. Bain, J. Huh, T. Han, and A. Zisserman, “*WhisperX: Time-Accurate Speech Transcription of Long-Form Audio*,” in **Proc. Interspeech**, 2023.
- [12] A. Dawar, “*Why Deep Learning is the Best Approach for Speech Recognition*,” Deepgram Blog, Jun. 2024.
- [13] M. Bain *et al.*, “*Conformer-1: A Robust Speech Recognition Model Trained on 650K Hours of Data*,” AssemblyAI Blog, Oct. 2022.
- [14] D. S. Park *et al.*, “*SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition*,” in **Proc. Interspeech**, 2019.

- [15] OpenAI, “*Introducing Whisper*,” OpenAI, Sep. 21, 2022. [Online]. Available: <https://openai.com/index/whisper/> (accessed Dec. 10, 2025).
- [16] Z. Chang *et al.*, “*A survey of LLM alignment: Instruction understanding, intention reasoning, and reliable dialogue generation*,” *Neurocomputing*, vol. 687, Art. no. 133629, 2026.
- [17] T. Bocklisch, T. Werkmeister, D. Varshneya, and A. Nichol, “*Task-Oriented Dialogue with In-Context Learning*,” Rasa, Feb. 15, 2024. [Online]. Available: <https://huggingface.co/papers/2402.12234> (accessed May 4, 2026).
- [18] Y. Choi, C. Li, Y. Yang, and Z. Jin, “*Agent-to-Agent Theory of Mind: Testing Interlocutor Awareness among Large Language Models*,”. Available: <https://huggingface.co/papers/2506.22957> (accessed May 4, 2026).
- [19] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, “*A Survey on Knowledge Graphs: Representation, Acquisition, and Applications*,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 494-514, Feb. 2022. [Online]. Available: <https://huggingface.co/papers/2002.00388> (accessed May 4, 2026).
- [20] A. A. Kamalipour, S. Asadi, and M. M. Amiri Chimeh, “*From vectors to knowledge graphs: A comprehensive analysis of modern retrieval-augmented generation architectures*,” *Comput. Sci. Rev.*, vol. 61, Art. no. 100925, Aug. 2026.
- [21] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, and S. Tang, “*Graph Retrieval-Augmented Generation: A Survey*,” arXiv preprint arXiv:2408.08921, Aug. 2024. [Online]. Available: **Error! Hyperlink reference not valid.** (accessed May 4, 2026).
- [22] T. Likhomanenko *et al.*, “*ChipChat: Low-Latency Cascaded Conversational Agent in MLX*,” in ASRU, Dec. 2025. [Online]. Available: <https://machinelearning.apple.com/research/chipchat> (accessed May 4, 2026).
- [23] D. Jacoby, T. Zhang, A. Mohan, and Y. Coady, “*Human Latency Conversational Turns for Spoken Avatar Systems*,” arXiv preprint arXiv:2404.16053, Apr. 2024. [Online]. Available: <https://huggingface.co/papers/2404.16053> (accessed May 4, 2026).
- [24] J. De Koninck, “*How Does Gamification Drive Engagement?*,” StriveCloud. [Online]. Available: <https://www.strivecloud.io/blog/how-gamification-drives-engagement> (accessed May 4, 2026).

Match Overview			×
3%			
< >			
1	arxiv.org Internet Source	<1%	>
2	core.ac.uk Internet Source	<1%	>
3	Submitted to Higher Ed... Student Paper	<1%	>
4	www.strivecloud.io Internet Source	<1%	>
5	Amir Abbas Kamalipou... Publication	<1%	>
6	ceo-bench.dave.engine... Internet Source	<1%	>
7	www.mdpi.com Internet Source	<1%	>
8	research.aalto.fi Internet Source	<1%	>
9	Submitted to University... Student Paper	<1%	>
10	Submitted to University... Student Paper	<1%	>

0% detected as AI ⓘ

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

☰ Submission Breakdown



0 AI-generated only

0%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased

0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.